

REMAINING SHELF LIFE ESTIMATION OF FRESH FRUITS AND VEGETABLES DURING THE TRANSPORTATION

A Project report submitted in partial fulfillment
for the award of the degree of

**MASTER OF COMPUTER APPLICATIONS
(2022-2024)**

Submitted by

**M. MANASA
(22G21F0061)**

Under the esteemed guidance of

Mrs. CH. RUSHYENDRA MANI
Assistant Professor



Department of Master of Computer Applications

**AUDISANKARA COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)**

(Accredited by NAAC A+)

Approved by AICTE, Affiliated to JNTU-A,
Ananthapuramu, Tirupati(Dt.), Andhra Pradesh
NH-5, Bypass Road, Gudur, Tirupati(Dt.)

www.audisankara.ac.in

2022-2024

AUDISANKARA COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)

(Accredited by NAAC A+)

Approved by AICTE, Affiliated to JNTUA,
Ananthapuramu, Tirupati(Dt.), Andhra Pradesh.



CERTIFICATE

This is to certify that the project report entitled **“REMAINING SHELF-LIFE ESTIMATION OF FRESH FRUITS AND VEGETABLES DURING THE TRANSPORTATION”** is the bonafide work done by me **M.MANASA, REGD.NO-22G21F0061** in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications**, from Jawaharlal Nehru Technological University Anantapur, Anantapuramu, during the year 2022-2024.

Project Guide

Mrs. CH.RUSHYENDRA MANI

Assistant Professor

Department of master of Computer Applications
AUDISANKARA COLLEGE OF ENGG & TECH.
GUDUR - TIRUPATI DISTRICT.

Head of the department

Prof. V. CHANDRASEKHAR

Associate Professor

Department of Master of Computer Applications
AUDISANKARA COLLEGE OF ENGG & TECH
GUDUR - TIRUPATI DISTRICT.

Submitted for the viva-voice examination held on _____

Internal Examiner

External Examiner

**AUDISANKARA COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)**

(Accredited by NAAC A+)

Approved by AICTE, Affiliated to JNTUA,
Ananthapuramu, Tirupati(DT), Andhra Pradesh.



DECLARATION

I, **Ms. M. MANASA, Regd.No-22G21F0061**, hereby declare that the Project Work entitled **“REMAINING SHELF-LIFE ESTIMATION OF FRESH FRUITS AND VEGETABLES DURING THE TRANSPORTATION”** done by us under the esteemed guidance of Assistant Professor **Mrs. RUSHYENDRA MANI**, and is submitted in partial fulfillment of the requirements for the award of the **Master’s of Degree in Computer Applications**.

Date:

Place:

Signature of the Candidate

M. MANASA
(22G21F0061)

ACKNOWLEDGEMENT

The satisfaction and elation that accompany the successful completion of any task would be incomplete without the mention of the people who have made it a possibility. It is my great privilege to express my gratitude and respect to all those who have guided me and inspired me during the course of this project work.

First and foremost, I express my sincere gratitude to our honorable chairman **Dr. VANKI PENCHALAIAH**, M.A.,M.L.,Ph.D., who provided all facilities and necessary encouragement during the course of study.

I extend my gratitude and sincere thanks to our beloved Director **Dr. A. MOHANBABU**, principal **Dr. J. RAJA MURUGADOSS**, for motivating and providing necessary infrastructure and permitting me to complete the project.

I would like to express the sense of gratitude towards our Head of the Department **Prof. V. CHANDRASEKHAR**, my internal guide **Mrs. CH.RUSHYENDRA MANI**, Assistant Professor and for their support and encouragement for completion of this project.

Finally, I express my sincere thanks to all the teaching and non-teaching staff that guided and helped me to complete the project work successfully.

ABSTRACT

During transportation, prediction of the Remaining Shelf-Life RSL of Fresh Fruits and Vegetables (FFVs) is critical for planning and quality cost estimation. The Internet of Things (IoT) enables measured environmental variables to be processed in real-time. However, there is a need for a validated, real-time computational method that translates environmental measurements to dynamic RSL estimates. Most existing generic RSL models for FFVs are qualitative, invasive, or static. This study establishes a generic RSL model for FFVs under dynamic and unplanned logistic conditions. The model is based on estimating the current rate of general decay based on the expected respiration rate of the product, and integrating the decay rate with respect to time. Its implementation is non-destructive, non-invasive, and does not require accelerated shelf-life experiments before deployment. In addition, since the original model is rather computationally intensive, a surrogate model was proposed to allow the model to be implemented in fast, real-time applications for 'Edge IoT.' Experimental validation of the model using three fresh products eg: strawberries, apricots, and spinach in a domestic refrigerator resulted in a maximum deviation of 1.3 days in prediction error using the original model and 2.95 days using the surrogate model. Nonetheless, the predictions made using either the original or surrogate models were statistically sound and not significantly different from the observed shelf lives of the samples, even at the 0.01 significance level.

INDEX

S.NO	CONTENTS	PAGE NO
	ABSTRACT	
1.	INTRODUCTION	1
2.	LITERATURE SURVEY	4
3.	SYSTEM ANALYSIS	9
3.1	EXISTING SYSTEM	9
3.2	PROPOSED SYSTEM	9
4.	SYSTEM REQUIREMENTS	11
4.1	SOFTWARE REQUIREMENTS	11
4.2	HARD WARE REQUIREMENTS	11
5.	SYSTEM STUDY	12
6.	SYSTEM DESIGN	13
6.1	SYSTEM ARCHITECTURE	22
6.2	INPUT AND OUTPUT REPRESENTATION	23
6.3	UML DIAGRAMS	24
7.	SYSTEM IMPLEMENTATION	33
8.	SYSTEM TESTING	53
9.	SCREENSHOTS	56
10.	CONCLUSION	73
11	REFERENCE	74

1. INTRODUCTION

ABOUT MACHINE LEARNING

Machine learning (ML) is the study of computer algorithms that improve automatically through experience. It is seen as a subset of artificial intelligence. Machine learning algorithms build a mathematical model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly programmed to do so. Machine learning algorithms are used in a wide variety of applications, such as email filtering and computer vision, where it is difficult or infeasible to develop conventional algorithms to perform the needed tasks.

Machine learning is closely related to computational statistics, which focuses on making predictions using computers. The study of mathematical optimization delivers methods, theory and application domains to the field of machine learning. Data mining is a related field of study, focusing on exploratory data analysis through unsupervised learning. In its application across business problems, machine learning is also referred to as predictive analytics.

Machine learning involves computers discovering how they can perform tasks without being explicitly programmed to do so. It involves computers learning from data provided so that they carry out certain tasks. For simple tasks assigned to computers, it is possible to program algorithms telling the machine how to execute all steps required to solve the problem at hand; on the computer's part, no learning is needed. For more advanced tasks, it can be challenging for a human to manually create the needed algorithms. In practice, it can turn out to be more effective to help the machine develop its own algorithm, rather than have human programmers specify every needed step.

The discipline of machine learning employs various approaches to help computers learn to accomplish tasks where no fully satisfactory algorithm is available. In cases where vast numbers of potential answers exist, one approach is to label some of the correct answers as valid. This can then be used as training data for the computer to improve the algorithm(s) it uses to determine correct answers. For example, to train a system for the task of digital character recognition, the MNIST dataset has often been used.

MACHINE LEARNING APPROACHES

Early classifications for machine learning approaches sometimes divided them into three broad categories, depending on the nature of the "signal" or "feedback" available to the learning system. These were:

Supervised Learning: The computer is presented with example inputs and their desired outputs, given by a "teacher", and the goal is to learn a general rule that maps inputs to outputs.

Unsupervised Learning: No labels are given to the learning algorithm, leaving it on its own to find structure in its input. Unsupervised learning can be a goal in itself (discovering hidden patterns in data) or a means towards an end (feature learning).

Reinforcement Learning: A computer program interacts with a dynamic environment in which it must perform a certain goal (such as driving a vehicle or playing a game against an opponent) As it navigates its problem space, the program is provided feedback that's analogous to rewards, which it tries to maximise.

Other approaches or processes have since developed that don't fit neatly into this three-fold categorisation, and sometimes more than one is used by the same machine learning system. For example topic modeling, dimensionality reduction or meta learning. [8] As of 2020, deep learning has become the dominant approach for much ongoing work in the field of machine learning.

ABOUT THE PROJECT

The waste of Fresh Fruits and Vegetables (FFVs) is a significant issue in food supply chain industries. Srivastava et al. reported that food retailers experience profit margin losses due to coldchain risks such as transport delays and breakdowns, temperature abuse, and cross-contamination. Food losses are a burden to society, too, with one-third of all edible human food wasted annually. It is well known that temperature variability is inevitable across the supply chain and that in the case of fresh products, degradation reactions are accelerated at temperatures above the recommended storage temperatures. Therefore, temperature variability must be accounted for when estimating the product's RSL at any point in the chain.

IoT makes it possible to measure environmental parameters such as temperature and humidity during transit and communicate these measurements in real-time to the cloud for quality monitoring and real-time fleet routing optimization.

A mathematical model that accepts the measured environmental parameters and outputs the RSL for a given product in real time is thus pivotal to the success of such advanced decision systems. Predicting the shelf-life of sealed, dried, highly processed, and chemically-preserved foods is well understood and established with simple and reliable mathematical models.

On the other hand predicting the shelf-life of unsealed, unprocessed, and biologically-active foods (such as fresh fruits and vegetables, milk, fish, and meat) shipped together with other types of fresh foods under the problem.

OBJECTIVE OF PROJECT

The objective generally involves developing a comprehensive model to predict airfare prices by leveraging machine learning techniques.

2. LITERATURE SURVEY

During circulation, it cannot be guaranteed that the product's recommended storage temperature requirements will always be met. Consequently, the problem of shelf-life reduction due to deviation from the recommended storage conditions has long been of interest to the food science community. There are five types of decay in relation to FFVs: Preharvest biological decay, senescence, microbiological decay, chemical deterioration, and physical degradation.

Despite the temperature and headspace gas composition being just two of many sources of degradation of fresh produce, they are still considered major predictors of post-harvest degradation and distribution losses. As a result, temperature and gas measurements can be used to provide an optimistic prediction of the RSL. All other sources of decay, such as vibration and moisture loss, can be ignored if they are not significantly different from the logistic conditions under which the initial shelf-life was estimated.

Hence, for all future discussions, we shall use the word 'decay' to refer to the amalgamation of all post-harvest, undesirable quality changes that are affected by temperature and/or atmospheric composition, namely: Senescence, chemical deterioration, and microbiological decay.

In their review, Hertog et al. categorized the modeling approaches for warehouse management into three groups, all of which use the measured environmental conditions along the cold chain. Namely, Statistical Process Control (SPC), specific quality attribute models, and generic shelf-life models. SPC models focus on monitoring and correcting environmental conditions, while specific attribute models monitor a single biochemical attribute over time. The most comprehensive approach is the generic shelf-life model, where the product's shelf-life is determined considering not only a limiting quality attribute of the product but also its respiration activity and moisture loss. We are interested in the second and third approaches, as they output an RSL estimate: An actionable piece of information based on which the storage, sales, and distribution of perishable products can be optimized, as opposed to a binary (Keep/Discard) recommendation from an SPC model.

Most well-known mathematical models of shelf-life traditionally fall into the second category (specific quality attribute models), consisting of a primary kinetic model and one or more secondary models. The primary model consists of one or more differential equations relating the rate of change of the food quality index to time, while a secondary model relates the rate of change of the food quality index to degradation factors other than time, such as temperature, pH, water activity, etc.

The evolution of the shelf-life (defined in terms of a food quality index) is found by integrating the differential equations in the primary and secondary models over time. To develop a kinetic model of food degradation, a measurable physical, biological, chemical, or sensory property of the food that strongly correlates with loss of quality and customer acceptance Q , is first defined, then experimentally measured for different storage temperatures and times. Afterward, the relationship between Q and storage time t at a constant temperature T is fitted with either a straight-line on a linear plot or a semi-log plot of the data points, whichever is the best fitting. The differential of this line is of either zero-order or first-order, based on which the overall kinetics of decay are classified as either zero-order or first-order kinetics, respectively, and the reaction rate constant k is found at that specific temperature T . The reaction rate constant k is often a function of the storage temperature T , thus the need for a secondary model to account for the effect of temperature on the rate constant k .

The Arrhenius relationship has been widely used to model the reaction rate constant, k with respect to T . Eventually, a specific quality attribute model considers a product to have reached its shelf-life when the quality index Q reaches a predefined acceptability limit.

Despite the overwhelming popularity of using traditional zero-, first-, and second-order kinetics to model degradation in fresh and horticultural products, asserted that most food degradation processes do not follow any standard kinetic model and that shelf-life estimation need not assume a kinetic model order a priori. Another drawback of using fixed-order kinetics is that the change in the limiting quality attribute is often due to multiple biochemical reactions which are not necessarily of the same order. Moreover, several models of different order might fit the data points equally well, resulting in the reaction order not being clearly defined and possibly, the selection of an inaccurate model that systematically over- or underestimates the shelf-life.

The equally popular Arrhenius model has been classically used as a secondary model for microbial growth and, consequently, food deterioration models. However, its widespread adoption has also been greatly criticized. It is perfectly appropriate for predicting the rate of reactions between gases in solutions, but “makes no sense at all in the description of microbial growth kinetics” [9, p. 932] for several reasons, including but not limited to: The temperature-independence of the activation energy, the model’s inability to account for optimal growth temperature, the irrelevance of the Universal Gas Constant R , and (in microbial growth models specifically), the cumbersome reference to a mol of bacteria (which would weigh 600,000 metric tons), and that it attempts to model the number of microbial cells or their growth rate, which is fundamentally different than modeling the concentration of a chemical reactant.

Furthermore, reviewed the appropriateness of the Arrhenius and two other widely-used secondary models, namely, the Eyring-Polanyi and WLF models, both of which were - like the Arrhenius model- originally derived for nonfood systems, and pointed out that a model's good fit alone does not assure its validity. They demonstrated that using those three classical models in the context of food deterioration is theoretically flawed and that their parameters should not be given any physical meaning.

Overall, determining kinetic model constants typically requires tedious accelerated shelf-life studies that must be conducted for each unique combination of food type, quality index, and packaging. Nonetheless, the best quality index to be used as a predictor of deterioration can be different for each product, and - for the same product- differ for isothermal versus non-isothermal conditions. For example, found that the shelf-life of fresh rocket leaves was limited by their appearance score at 5°C, but by their ascorbic acid content under dynamic temperature conditions. Generally, the use of specific quality attribute models - as defined by also entails the determination of the initial value of the selected food quality index: This may not always be feasible and expedient. Although the index might be a quality that can be measured instantly and non-destructively, such as the color of the product, which can be measured easily with a portable spectrophotometer, it may very well be a quality that takes much longer to measure, requires a destructive or invasive procedure, relatively expensive equipment.

A specialized lab environment such as microbial counting, for which a sample needs to be incubated for 1-2 days in a sterilized environment. Without the initial value of the food quality index, the applicability of many kinetic models in the FFV sector would be limited.

Equally important practical challenges also include the real-time determination of FFV pulp temperature nondestructively, as well as exogenous and endogenous factors that affect the deterioration rate of FFVs besides temperature, namely: O₂ and CO₂ concentrations, and the respiratory climacteric phases characteristic of climacteric fruits, both of which cannot be captured by a single secondary model for temperature.

The above reasons have instigated research into other smarter methods of food quality monitoring, such as Time. Temperature Indicators (TTIs), which are non-destructive and responsive to the product's surface temperature rather than the ambient temperature.

TTIs are small, physical devices in the form of small stickers placed on food packages that change color irreversibly if a threshold temperature is exceeded (partial-history TTIs) or when the cumulative temperature history renders the product unsafe (full-history TTIs).

As they only provide visual cues, they do not provide quantitative information about the RSL. A recent review of the commercially-available TTIs shows that all but one are only qualitative indicators. Moreover, the only quantitative indicator simply indicates cumulative temperature exposure and not necessarily the quality or freshness of the product. Without quantitative shelf-life estimates available in real-time, third-order cold supply chain logistics (where actionable insights and decisions are produced and altered in real-time) cannot be implemented. La Scalia et al. developed a battery-powered Smart Logistic Unit (SLU) that measures temperature, CO₂, relative humidity (RH), and the concentration of Volatile Organic Compounds (VOCs), contains a GPS module, and has network connectivity to post measurements to a database.

A shelf-life model was formulated and tested on strawberries based on the linear correlation between VOC level and microbiological growth. In practice, however, many products are shipped in air-tight packaging, making it impossible for such a device to detect the CO₂ or VOCs emitted by such products. In addition, several storage-compatible products are often shipped in a single.

The shelf-life estimates from each of the four models were compared to the true shelf-life determined based on microbiological testing. Although they found the kinetic Arrhenius shelf-life model to have the smallest percent error, one cannot conclude that the kinetic Arrhenius model is the best overall, as it is the model which takes the fewest external factors into account: Temperature only. Furthermore, the theoretical soundness of using the Arrhenius model for the shelf-life prediction of food remains a highly divisive issue.

Not to mention that air-tight packaging and/or mixing different products in the same storage compartment would render inapplicable any models relying on the direct measurement of gas concentrations. Therefore, a calculated-CO₂-based model seems the most digestible in theory and expedient practice, but structuring of that model lacks at least two respects: 1) Aerobic respiration was assumed throughout, which may not always be the predominant mode of respiration under modified atmospheres, and 2) A constant respiratory quotient was assumed- whereas it is generally a function of temperature, storage time, and the concentrations of CO₂ and O₂. To summarize, third-order cold supply chain logistics require advancing generic shelf-life models, rather than specific quality attribute models.

Despite being the most popular, specific quality attribute models are flawed in their theoretical bases, are time-consuming (and sometimes too expensive) to apply in the last-mile delivery of FFVs, and, most importantly, fail to capture the effects of dynamic environmental factors other than temperature.

Generic shelflife models, in essence, recognize and acknowledge the importance of environmental effects other than temperature, such as VOCs, vibration, moisture loss, and current respiration activity.

However, based on our literature review, the following challenges hinder the real-time application of generic shelf-life models in the last-mile delivery of FFVs:

- The dynamic formation of a local, modified atmosphere due to airtight packaging.
- The dynamic formation of a local, modified atmosphere due to airtight packaging.
- When measured CO₂ and VOCs are used to track the respiration activity, how would such a model handle a transportation scenario where multiple species of products are being shipped together in the same compartment and sharing the same headspace?
- The onset of anaerobic respiration in each species when CO₂ gas concentrations reach their maximum tolerable CO₂ limits.
- The changes in respiration rate that occur due to the onset of anaerobic respiration and due to different climacteric phases.
- Estimation of the product's pulp temperature dynamically and non-destructively, since the product's respiration rate is a function of the product's pulp temperature rather than its ambient temperature.

Hence, the objective of this study is to establish a generic shelf-life model for FFVs under dynamic logistic conditions that addresses the above five challenges. The model proposed in this paper has been designed such that the effects of instantaneous temperature and CO₂ gas concentration in the local atmosphere are captured without the need for directly measuring the latter. In addition, the model can be applied independently of whether the product is in standard air or a modified atmosphere, shipped alone or with other storage compatible products.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

- During circulation, it cannot be guaranteed that the product's recommended storage temperature requirements will always be met.
- Consequently, the problem of shelf-life reduction due to deviation from the recommended storage conditions has long been of interest to the food science community.
- To develop a kinetic model of food degradation, a measurable physical, biological, chemical, or sensory property of the food that strongly correlates with loss of quality and customer acceptance, Q , is first defined, then experimentally measured for different storage temperatures and times.
- Developed a battery-powered Smart Logistic Unit (SLU) that measures temperature, CO₂, relative humidity (RH), and the concentration of Volatile Organic Compounds (VOCs), contains a GPS module, and has network connectivity to post measurements to a database.

DISADVANTAGES OF EXISTING SYSTEM

- The dynamic formation of a local, modified atmosphere due to airtight packaging
- The changes in respiration rate that occur due to the onset of anaerobic respiration and due to different climacteric phases
- Estimation of the product's pulp temperature dynamically and non-destructively, since the product's respiration rate is a function of the product's pulp temperature rather than its ambient temperature.

3.2 PROPOSED SYSTEM

- The proposed system presents a new, real-time, non-destructive, highly generalizable computational method to predict the remaining shelf-life of FFVs under unplanned logistic conditions.
- The method uses real-time measurements of temperature and humidity and most significantly - precludes the need for characterization pre-tests as it is initialized with only well-documented and easily accessible thermophysical properties of the product.
- Moreover, the proposed model has been validated with three types of FFVs under .

ADVANTAGES OF PROPOSED SYSTEM

- The model proposed in this project has been designed such that the effects of instantaneous temperature and CO₂ gas concentration in the local atmosphere are captured without the need for directly measuring the latter.
- In addition, the model can be applied independently of whether the product is in standard air or a modified atmosphere, shipped alone or with other storage compatible products.

4. SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

- Operating System : Windows 1064 bit
- Programming language : Python
- Front End : HTML, CSS
- Back End : MySQL 5.0
- Framework : Django
- SERVER : XAMPP

4.2 HARDWARE REQUIREMENTS

- Processor : I3
- RAM : 4 GB
- Hard Disk : 500GB

5. SYSTEM STUDY

FEASIBILITY STUDY

TECHNICAL FEASIBILITY

Technically feasible, Estimating the remaining shelf-life of fresh fruits and vegetables during transportation is technically feasible with current technologies. It requires a combination of IoT sensors, data analytics, machine learning, and cloud computing. By implementing such a system, it is possible to reduce food waste, ensure better quality of produce upon arrival, and optimize the supply chain operations.

ECONOMIC FEASIBILITY

The economic feasibility of estimating the remaining shelf-life of fresh fruits and vegetables during transportation involves evaluating the costs and benefits associated with implementing such a system. This estimation can help in reducing food waste, improving supply chain efficiency, and ensuring better quality products reach consumers.

OPERATIONAL FEASIBILITY

This system is being automated on the request of the technical department of our company. This new system meets their requirement and covers all aspects required much better than the old manual system. Most of the people involved in this branch are computer literate and do not need much training if this system is implemented. Hence it is operationally feasible.

6. SYSTEM DESIGN

The most creative and challenging phase of the life cycle is system and design. The term design describes a final system and the process by which it is developed. It refers to the technical specifications that will be applied in implementing the candidate system. The design may be defined as “the process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient details to permit its physical realization”.

The design's goal is how the output is to be produced and in what format samples of the output and input are also presented. Second input data and database files have to be designed to meet the requirements of the proposed output. The processing phase is handled through the program construction and testing. Finally details related to justification of the system and an estimate of the impact of the candidate system on the users and the organization are documented and evaluated by management as a step toward implementation.

The importance of software design can be stated in a single word “**Quality**”. Design provides us with representation of software that can be assessed for quality. Design is the only way that we can accurately translate a customer's requirements into a finished software product or system without design we risk building an unstable system, that might fail if small changes are made or may be difficult to test, or one whose quality can't be tested. So it is an essential phase in the development of a software product.

SDLC METHODOLOGY

The document plays a vital role in the development of life cycle (SDLC) as it describes the complete requirement of the system. It means for use by developers and will be the basic during testing phase. Any changes made to the requirements in the future will have to go through formal change approval process.

SPIRAL MODEL was defined by Barry Boehm in his 1988 article, “A spiral model of Software Development and Enhancement.

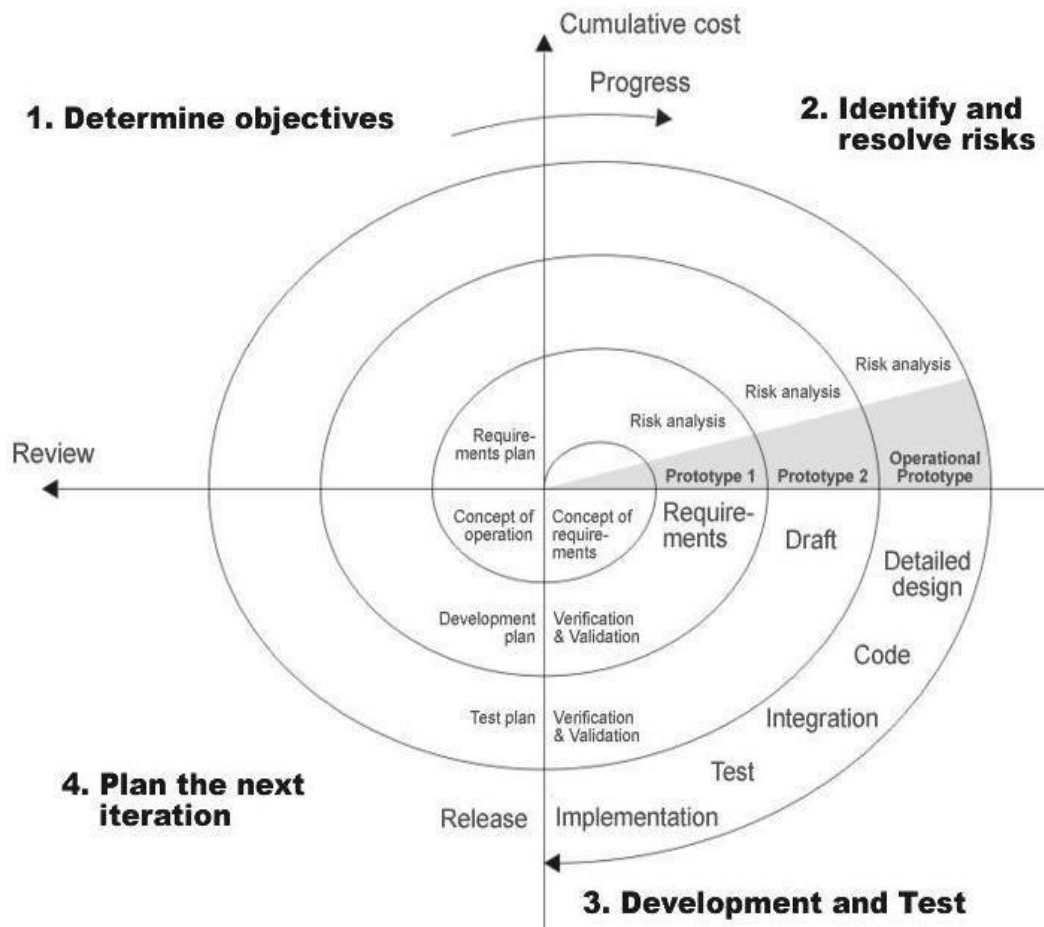


Fig: SDLC Methodology

STAGES IN SDLC

- Requirement Gathering
- Analysis
- Designing
- Coding
- Testing
- Maintenance

REQUIREMENTS GATHERINGSTAGE

The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements. These requirements define the major functions of the intended application, define operational data areas and reference data areas, and define the initial data entities. Major functions include critical processes to be managed, as well as mission critical inputs, outputs and reports.

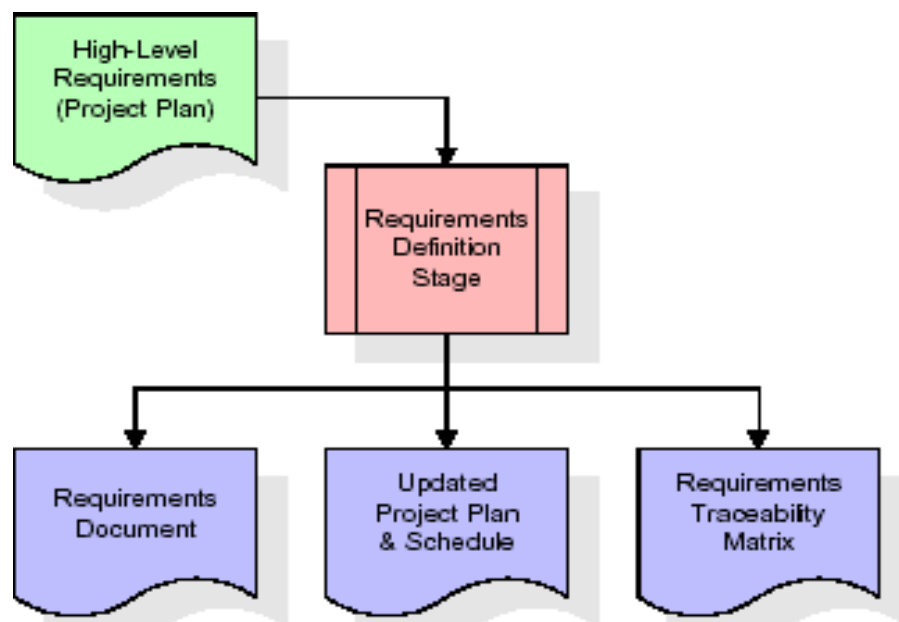


Fig: Requirements Gathering stage

These requirements are fully described in the primary deliverables for this stage: the Requirements Document and the Requirements Traceability Matrix (RTM). The requirements document contains complete descriptions of each requirement, including diagrams and references to external documents as necessary. Note that detailed listings of database tables and fields are *not* included in the requirements document.

The title of each requirement is also placed into the first version of the RTM, along with the title of each goal from the project plan.

The purpose of the RTM is to show that the product components developed during each stage of the software development lifecycle are formally connected to the components developed in prior stages.

In the requirements stage, the RTM consists of a list of high-level requirements, or goals, by title, with a listing of associated requirements for each goal, listed by requirement title. In this hierarchical listing, the RTM shows that each requirement developed during this stage is formally linked to a specific product goal.

- Feasibility study is all about identification of problems in a project.
- No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.
- Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.

ANALYSIS STAGE:

The planning stage establishes a bird's eye view of the intended software product, and uses this to establish the basic project structure, evaluate feasibility and risks associated with the project, and describe appropriate management and technical approaches.

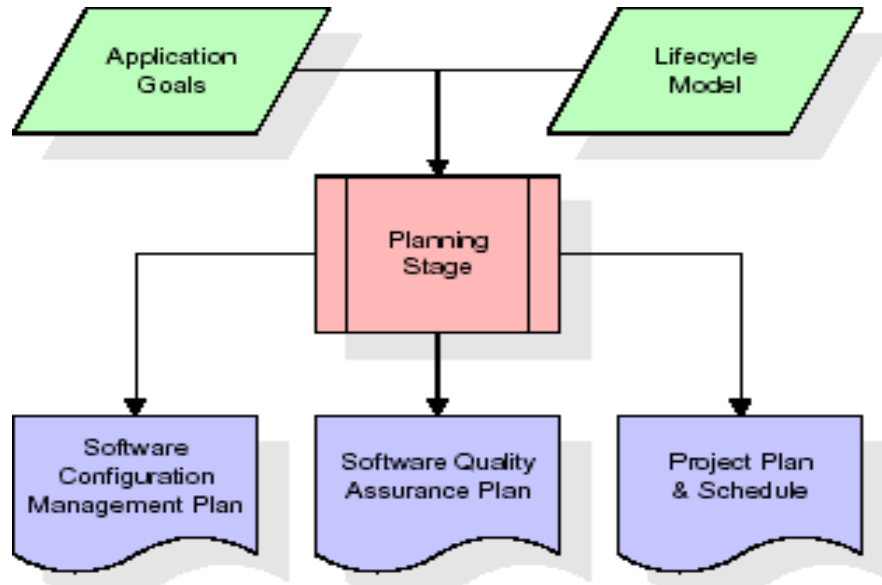


Fig: Analysis Stage

The most critical section of the project plan is a listing of high-level product requirements also referred to as goals. All of the software product requirements to be developed during the requirements definition stage flow from one or more of these goals. The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included. The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high level estimates of effort for the out stages.

The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements will be produced as a result of interviews, workshops, and/or prototype efforts. Design elements describe the desired software features in detail, and generally include functional hierarchy diagrams, screen layout diagrams, tables of business rules, business process diagrams, pseudo code, and a complete entity-relationship diagram with a full data.

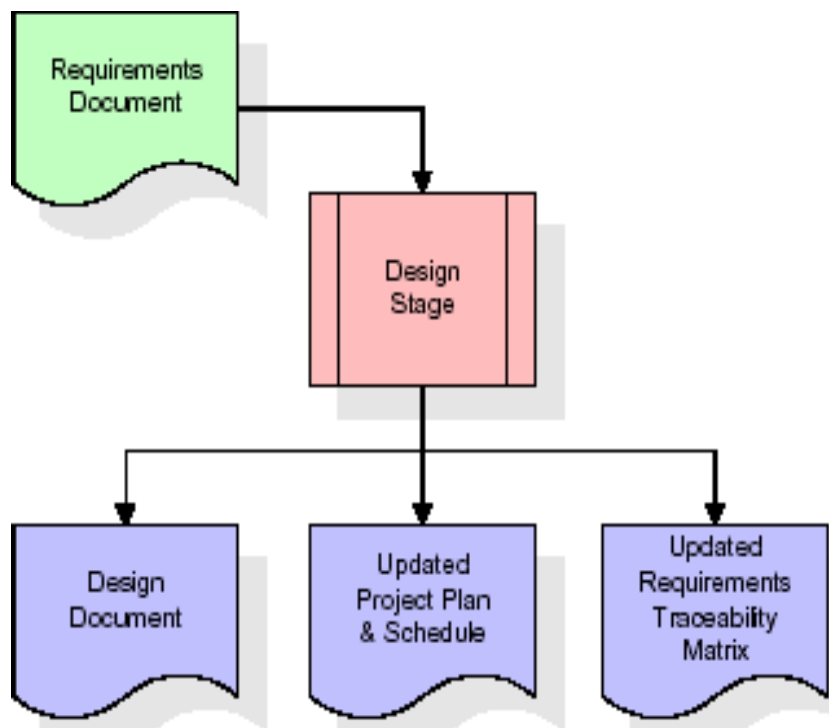


Fig: Analysis Stage2

When the design document is finalized and accepted, the RTM is updated to show that each designelement is formally associated with a specific requirement. The outputs of the design stage are thedesign document, an updated RTM, and an updated project plan.The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts will be produced.Software artifacts include but are not limited to menus, dialogs, data management forms, data reporting formats, and specialized procedures and functions. Appropriate test cases will bedeveloped for each set of functionally related software artifacts, and an online help system will bedeveloped to guide users in their interactions with the software.

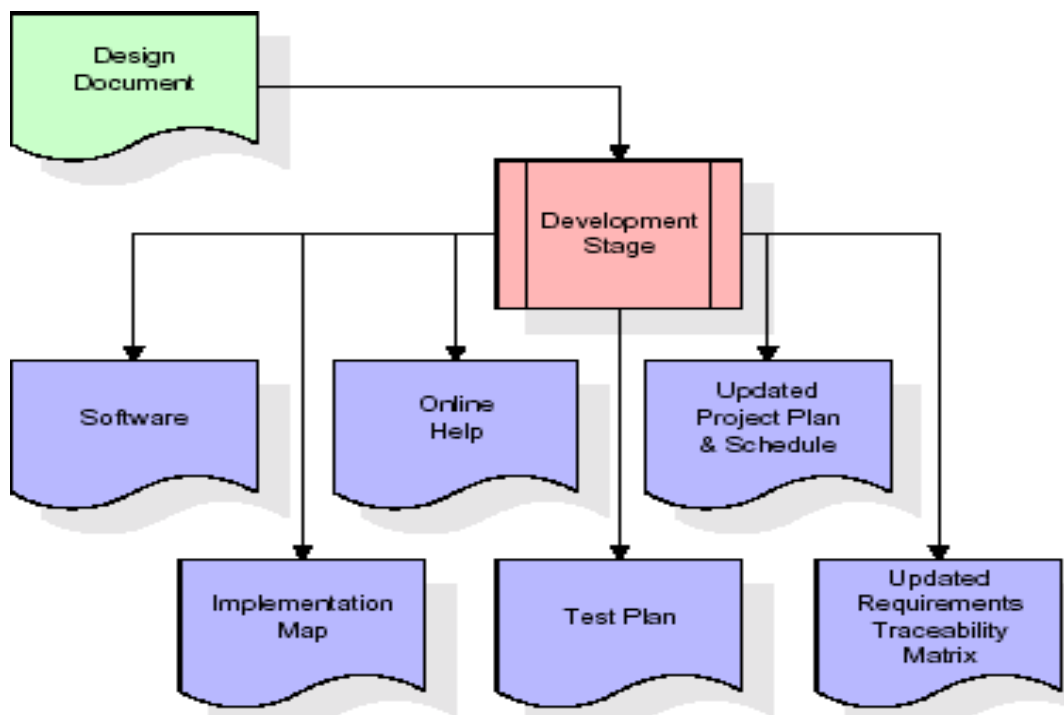


Fig: Analysis Stage3

The RTM will be updated to show that each developed artifact is linked to a specific designelement, and that each developed artifact has one or more corresponding test case items. At this point, the RTM is in its final configuration. The outputs of the development stage include a fully functional set of software that satisfies the requirements and design elements previously documented, an online help system that describes the operation of the software.

An implementation map that identifies the primary code entry points for all major system functions, a test plan that describes the test cases to be used to validate the correctness and completeness of the software, an updated RTM, and an updated project plan.

INTEGRATION & TEST STAGE

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite confirms a robust and complete migration capability. During this stage, reference data is finalized for production use and production users are identified and linked to their appropriate roles.

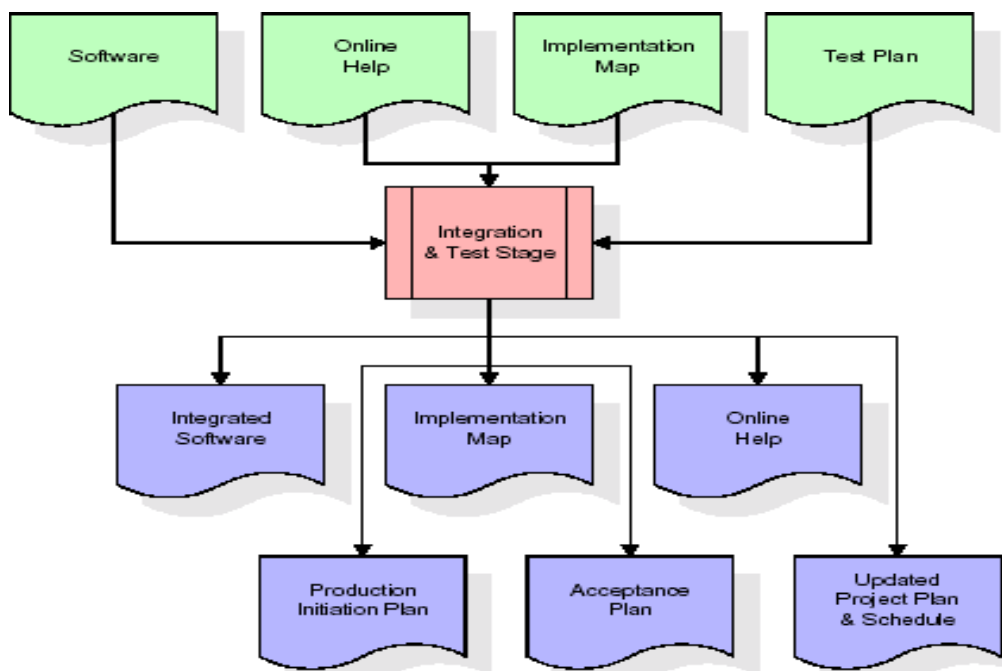


Fig: Integration Test Stage

The outputs of the integration and test stage include an integrated set of software, an online help system, an implementation map, a production initiation plan that describes reference data and production users, an acceptance plan which contains the final suite of test cases, and an updated project plan.

INSTALLATION & ACCEPTANCE TEST

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loaded onto the production server. prerequisite to acceptance of the softwar.

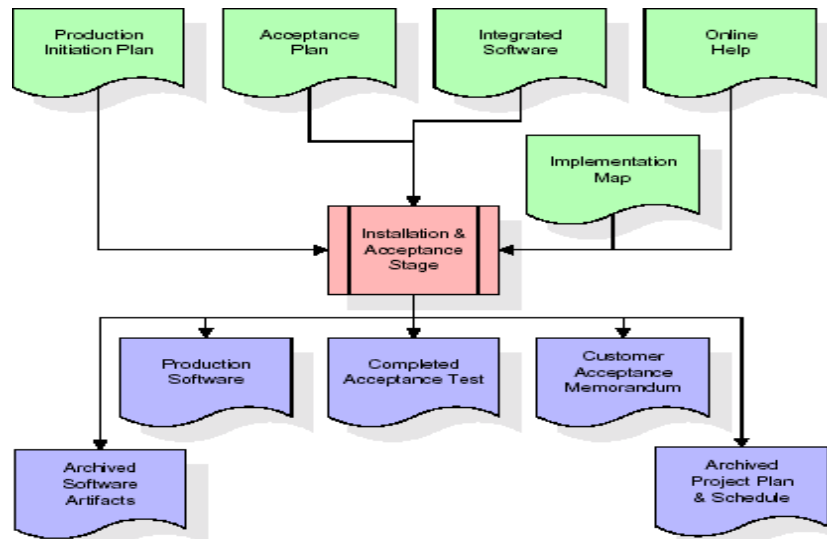


Fig: Installation Acceptance Test

The primary outputs of the installation and acceptance stage include a production application, a completed acceptance test suite, and a memorandum of customer acceptance of the software. Finally, the PDR enters the last of the actual labor data into the project schedule and locks the project as a permanent project record. At this point the PDR "locks" the project by archiving all software items, the implementation map, the source code, and the documentation for future reference.

MAINTENANCE

Outer rectangle represents maintenance of a project, Maintenance team will start with requirement study, understanding of documentation later employees will be assigned work and they will undergo training on that particular assigned category.

6.1 SYSTEM ARCHITECTURE

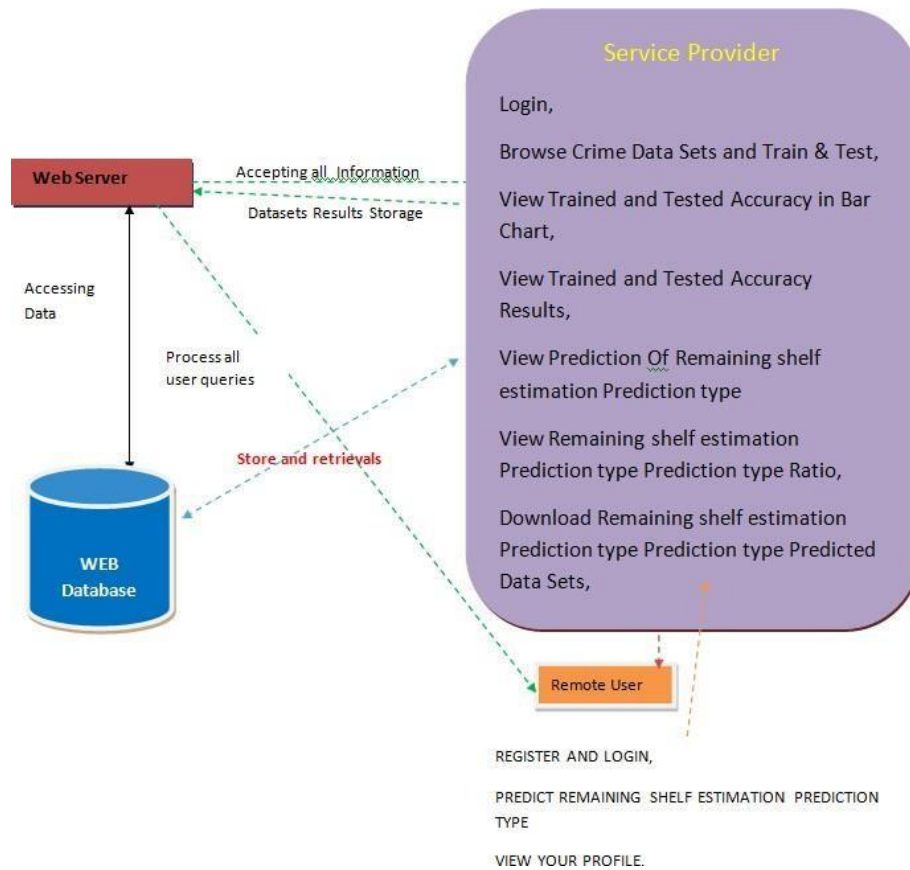


Fig: System Overview

6.2 INPUT & OUTPOUT REPRESENTETION

INPUT DESIGN

Input design includes data mediums used for inputting data and validations that are to be done during data entry. Different messages regarding data are given to guide users during data entry. Validation checks are done for each input.

Data entry screens are designed so that the system interacts with the user in providing an effective dialogue. Fields in the screen are logically arranged to help the user.

The design is the process of converting the user-originated inputs into a compute-based format. The goal of the input design is to make the data entry easier, logical and free from error. Errors in the input data are controlled by input design.

The application has been developed in a user-friendly manner. The windows have been designed in such a way that during the processing the cursor is placed in the position where the data must be entered. If any of the data going into the system is wrong then the process and output will magnify these errors.

The decisions made during design of input are:

- To achieve the highest possible level of accuracy.
- To provide a list of possible choices and help while accepting the input for an important field wherever possible outputs from computer system are required primarily to communicate the results of processing to the users. They are also used to provide a permanent copy of these results for later consultation/verification.

OUTPUT DESIGN

Output refers to the results and information that are generated by the system. Output is the main reason for developing the system and based on this, the usefulness and applicability of system are evaluated.

Outputs from computer systems are required primarily to communicate the results of processing to users. Efficiently designed outputs enhance the understandability of the information. Internal outputs, whose destination is within the organization and which require careful design because they are the user's main interface with the computer.

6.3 UML DIAGRAMS

The overall logical structure of a database can be expressed graphically by an **E-R diagram**. The relative simplicity and pictorial clarity of this diagramming technique may well account in large part for the widespread use of the E-R model. Such a diagram consists of the following major components.

Rectangles: Represent Entity Sets.

Ellipses: Represent attributes.

Diamonds: Represent relationship sets

Lines: Link attributes to entity sets and entity sets

DATA FLOW DIAGRAM

A graphical tool used to describe and analyze the movement of data through a system manual or automated including the process, stores of data, and delays in the system.

Data Flow Diagrams are the central tool and the basis from which other components are developed.

The transformation of data from input to output, through processes, may be described logically and independently of the physical components associated with the system. The DFD is also known as a data flow graph or a bubble chart.

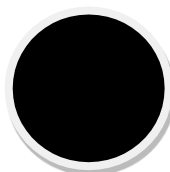
DFDs are the model of the proposed system. They clearly should show the requirements on which the new system should be built.

Later during design activity this is taken as the basis for drawing the system's structure charts. The Basic Notation used to create a DFD's are as follows:

1. Dataflow: Data move in a specific direction from an origin to a destination.



2. Process: People, procedures, or devices that use or produce (Transform) Data. The physical component is not identified.



3. Source: External sources or destination of data, which may be People, programs, organizations.



4.Data Store: Here data are stored or referenced by a process in the System.



UML DIAGRAMS:

Unified Modeling Language is a language available to perform modeling of software. A model is simplification of reality. A model provides the blue print of the system, model encompasses detailed plans.

BUILDING BLOCKS OF THE UML

The vocabulary of the UML encompasses three kinds of building blocks.

1. Things
2. Relationship
3. Diagram

THINGS IN THE UML

Things are the abstractions that are first-class citizen in a model. There are four kinds of things in the UML.

1. Structure things.
2. Behavioral things.
3. Grouping things.
4. Annotational things.

RELATIONSHIPS IN THE UML

Things can be connected to logically or physically with the help of relationship in object oriented modeling. These are four kinds of relationships in the UML.

1. Dependency.
2. Association.
3. Generalization.
4. Realization.

DIAGRAMS IN THE UML

A diagram is a graphical representation of a set of elements. There are nine kinds of diagrams in the UML.

1. Class diagram.
2. Object diagram
3. Usecase diagram.
4. Sequence diagram.
5. Collaboration diagram.
6. Activity diagram.
7. Component diagram.
8. State Chart diagram
9. Deployment diagram.

USECASE DIAGRAM

Use case diagram shows a set of usecases and actors (a special kind of class) and their relationship. Usecase diagrams address the static usecase view of a system.

These diagrams are especially important in organizing and modeling the behavioral of a system both sequence and collaboration diagrams are kind of interaction diagram.

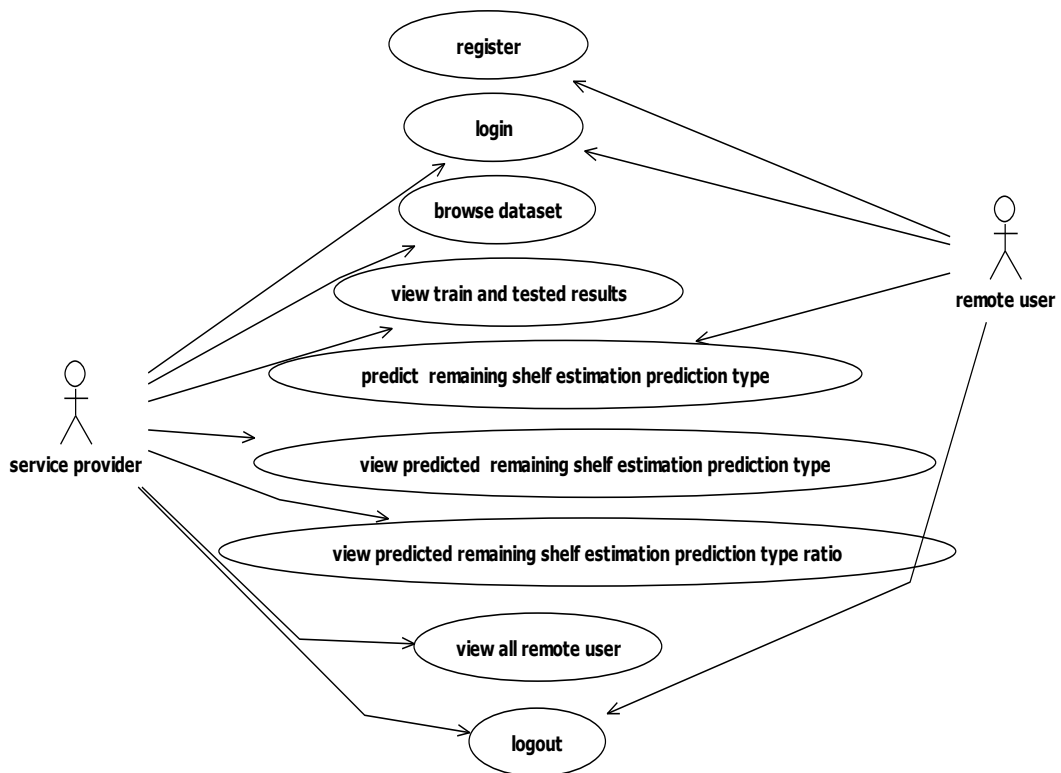


Fig: Use-case Diagram

CLASS DIAGRAM:

Class diagrams are one of the foremost common diagrams employed in UML. A class diagram consists of classes, interfaces, associations and collaborations. Class diagrams primarily represent the static structure of a system that is static in nature. A class diagram is employed in a very class diagram to represent the concurrency of the system.

A class diagram represents the static orientation of a system. Therefore it's usually used for development purpose. This can be the foremost wide used diagram at the time of system construction.

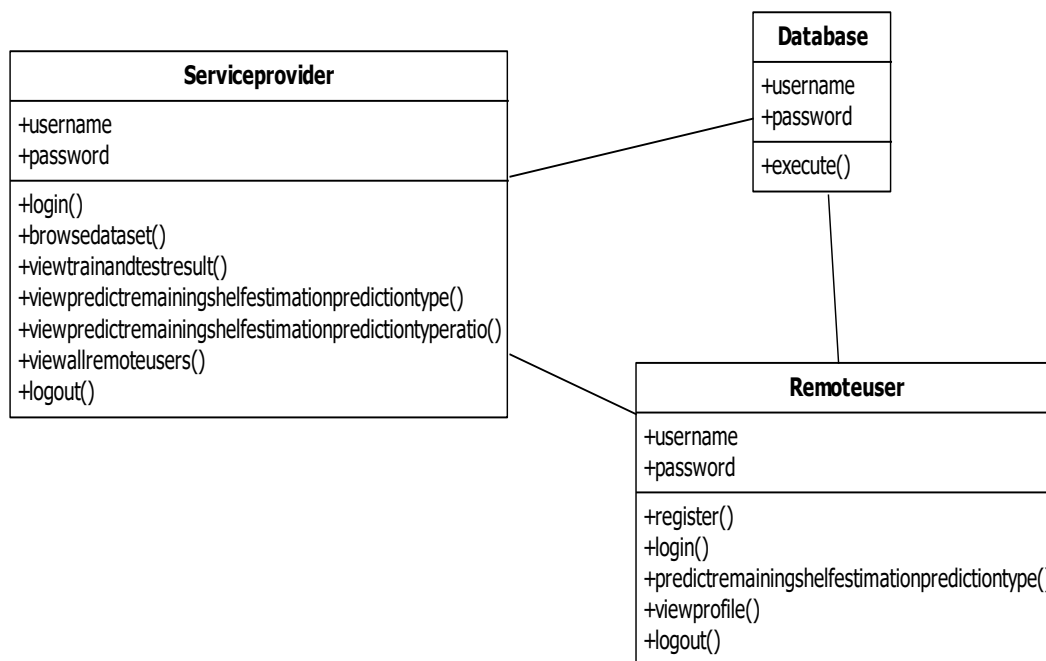


Fig: Class Diagram

SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a MessageSequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

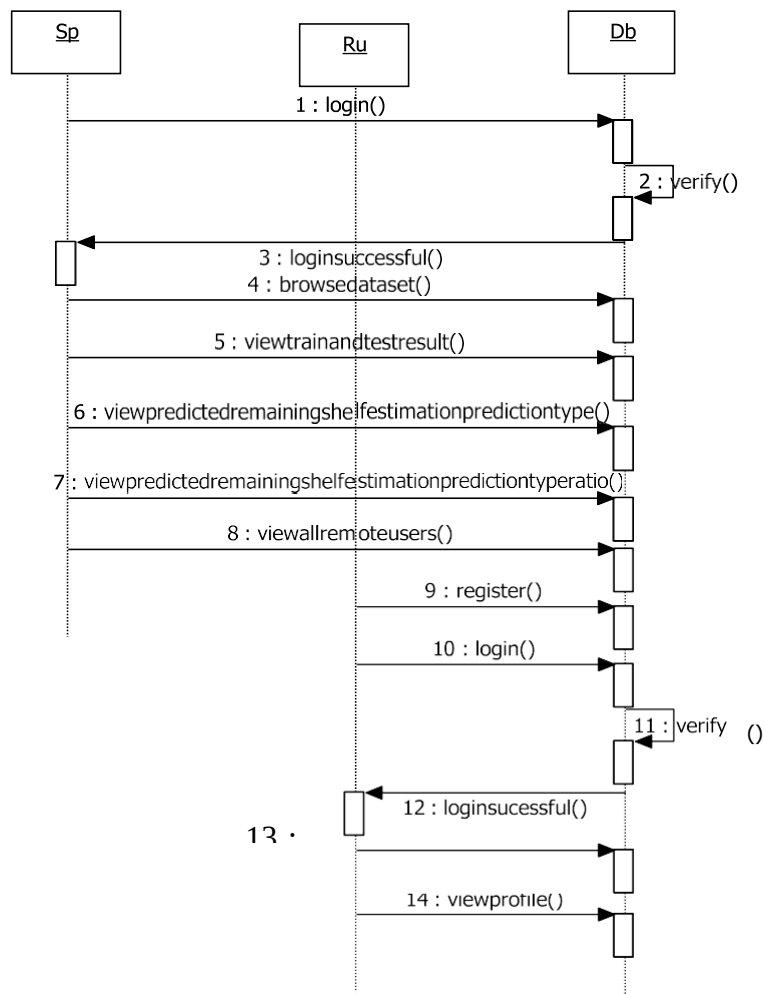


Fig: Sequence Diagram

ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control

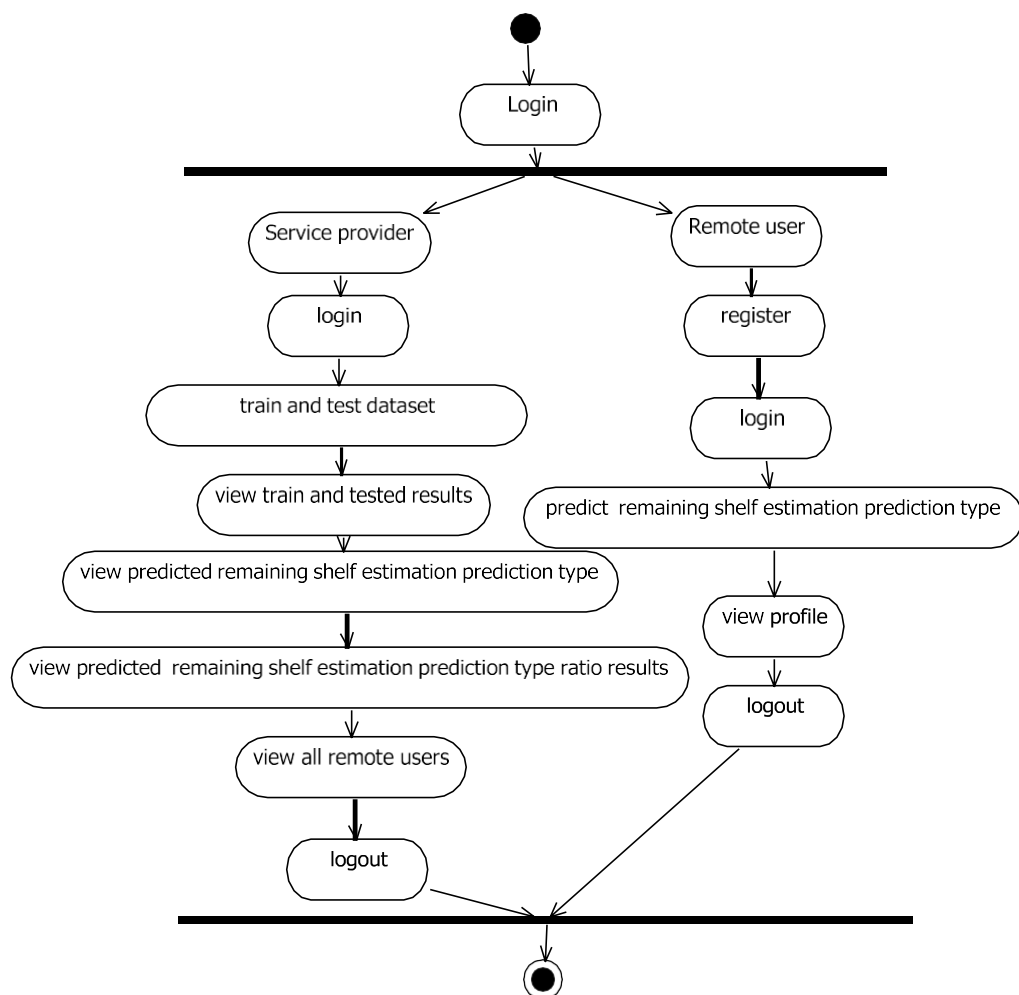


Fig: Activity Diagram

MODULE DESCRIPTION:

SERVICE PROVIDER

- In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as Browse the dataset, view trained and tested results, view predicted remaining shelf estimation prediction type, view predicted remaining shelf estimation prediction ratio and view all remote users

TRAIN AND TEST MODEL

- In this module, the service provider split the Used dataset into train and test data of ratio 70 % and 30 % respectively. The 70% of the data is consider as train data which is used to train the model and 30% of the data is consider as test which is used to test the model

REMOTE USER

- In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database.
- After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like predict remaining shelf estimation prediction type, View Your Profile.

CLASSIFICATION

- In this module, user enter data and classify them using tested machine learning models.

ALGORITHM

SUPPORT VECTOR MACHINE

- In machine learning, support-vector machines (SVMs, also support-vector networks) are supervised learning models with associated learning algorithms that analyze data for classification and regression analysis. An SVM training algorithm builds a model that assigns new examples to one category or the other, making it a non-probabilistic binary linear classifier.

LOGISTIC REGRESSION

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**

Naïve Bayes

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems.
- It is mainly used in text classification that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object

7. SYSTEM IMPLEMENTATION

This mapping permits forward engineering: The generation of code from a UML model into a programming language. The reverse is also possible: You can reconstruct a model from an implementation back into the UML. Reverse engineering is not magic. Unless you encode that information in the implementation, information is lost when moving forward from models to code. Reverse engineering thus requires tool support with human intervention.

TECHNOLOGY DESCRIPTION

INTRODUCTION TO PYTHON

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly, procedural), object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library. Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0, released in 2000, introduced features like list comprehensions and a garbage collection system capable of collecting reference cycles. Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3. The Python 2 language, i.e. Python 2.7.x, was officially discontinued on 1 January 2020 (first planned for 2015) after which security patches and other improvements will not be released for it.[32][33] With Python 2's end-of-life, only Python 3.5.x and later are supported. Python interpreters are available for many operating systems

SYNTAX AND SEMANTICS

Python is meant to be an easily readable language. Its formatting is visually uncluttered, and it often uses English keywords where other languages use punctuation. Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are optional. It has fewer syntactic exceptions and special cases than C or Pascal.

INDENTATION

Main article: Python syntax and semantics § Indentation

Python uses whitespace indentation, rather than curly brackets or keywords, to delimit blocks. An increase in indentation comes after certain statements; a decrease in indentation signifies the end of the current block. Thus, the program's visual structure accurately represents the program's semantic structure. This feature is sometimes termed the off-side rule, which some other languages share, but in most languages indentation doesn't have any semantic meaning.

STATEMENTS AND CONTROL FLOW

Python's statements include (among others):

The assignment statement (token '=', the equals sign). This operates differently than in traditional imperative programming languages, and this fundamental mechanism (including the nature of Python's version of variables) illuminates many other features of the language. Assignment in C, e.g., `x = 2`, translates to "typed variable name `x` receives a copy of numeric value 2". The (right-hand) value is copied into an allocated storage location for which the (left-hand) variable name is the symbolic address. The memory allocated to the variable is large enough (potentially quite large) for the declared type. In the simplest case of Python assignment, using the same example, `x = 2`, translates to "(generic) name `x` receives a reference to a separate, dynamically allocated object of numeric (int) type of value 2." This is termed binding the name to the object. Since the name's storage location doesn't contain the indicated value, it is improper to call it a variable.

Names may be subsequently rebound at any time to objects of greatly varying types, including strings, procedures, complex objects with data and methods, etc. Successive assignments of a common value to multiple names, e.g., `x = 2; y = 2; z = 2` result in allocating storage to (at most) three names and one numeric object, to which all three names are bound. Since a name is a generic reference holder it is unreasonable to associate a fixed data type with it. However at a given time a name will be bound to some object, which will have a type; thus there is dynamic typing.

- The if statement, which conditionally executes a block of code, along with else and elif (a contraction of else-if).
- The for statement, which iterates over an iterable object, capturing each element to a local variable for use by the attached block.
- The while statement, which executes a block of code as long as its condition is true.

- The class statement, which executes a block of code and attaches its local namespace to a class, for use in object-oriented programming.
- The def statement, which defines a function or method.
- The with statement, from Python 2.5 released in September 2006, which encloses a code block within a context manager (for example, acquiring a lock before the block of code is run and releasing the lock afterwards, or opening a file and then closing it), allowing Resource Acquisition Is Initialization (RAII)-like behavior and replaces a common try/finally idiom.
- The break statement, exits from the loop.
- The continue statement, skips this iteration and continues with the next item.
- The pass statement, which serves as a NOP. It is syntactically needed to create an empty code block.
- The assert statement, used during debugging to check for conditions that ought to apply.
- The yield statement, which returns a value from a generator function. From Python 2.5, yield is also an operator. This form is used to implement coroutines.

The import statement, which is used to import modules whose functions or variables can be used in the current program. There are three ways of using import: `import <module name> [as <alias>]` or `from <module name> import *` or `from <module name> import <definition 1> [as <alias 1>], <definition 2> [as <alias 2>],`

The print statement was changed to the print() function in Python 3.

Python does not support tail call optimization or first-class continuations, and, according to Guido van Rossum, it never will. However, better support for coroutine-like functionality is provided in 2.5, by extending Python's generators. Before 2.5, generators were lazy iterators; information was passed unidirectionally out of the generator. From Python 2.5, it is possible to pass information back into a generator function, and from Python 3.3, the information can be passed through multiple stack levels.

EXPRESSIONS

Some Python expressions are similar to languages such as C and Java, while some are not: Addition, subtraction, and multiplication are the same, but the behavior of division differs.

There are two types of divisions in Python.

In Python, `==` compares by value, versus Java, which compares numerics by value and objects by reference. (Value comparisons in Java on objects can be performed with the `equals()` method.) Python's `is` operator may be used to compare object identities (comparison by reference). In Python, comparisons may be chained, for example `a <= b <= c`.

Python uses the words `and`, `or`, `not` for its boolean operators rather than the symbolic `&&`, `||`, `!` used in Java and C.

Python has a type of expression termed a list comprehension. Python 2.4 extended list comprehensions into a more general expression termed a generator expression.

Anonymous functions are implemented using lambda expressions; however, these are limited in that the body can only be one expression.

Conditional expressions in Python are written as `x if c else y` (different in order of operands from the `c ? x : y` operator common to many other languages).

Python makes a distinction between lists and tuples. Lists are written as `[1, 2, 3]`, are mutable, and cannot be used as the keys of dictionaries (dictionary keys must be immutable in Python). Tuples are written as `(1, 2, 3)`, are immutable and thus can be used as the keys of dictionaries, provided all elements of the tuple are immutable. The `+` operator can be used to concatenate two tuples, which does not directly modify their contents, but rather produces a new tuple containing the elements of both provided tuples. Thus, given the variable `t` initially equal to `(1, 2, 3)`, executing `t = t + (4, 5)` first evaluates `t + (4, 5)`, which yields `(1, 2, 3, 4, 5)`, which is then assigned back to `t`, thereby effectively "modifying the contents" of `t`, while conforming to the immutable nature of tuple objects. Parentheses are optional for tuples in unambiguous contexts.

Python features sequence unpacking wherein multiple expressions, each evaluating to anything that can be assigned to (a variable, a writable property, etc.), are associated in the identical manner to that forming tuple literals and, as a whole, are put on the left hand side of the equal sign in an assignment statement.

The statement expects an iterable object on the right hand side of the equal sign that produces the same number of values as the provided writable expressions when iterated through, and will iterate through it, assigning each of the produced values to the corresponding expression on the left.

Python has a "string format" operator `%`. This function is analogous to `printf` format strings in C, e.g. `"spam=%s eggs=%d" % ("blah", 2)` evaluates to `"spam=blah eggs=2"`.

In Python 3 and 2.6+, this was supplemented by the `format()` method of the `str` class, e.g. `"spam={0} eggs={1}".format("blah", 2)`. Python 3.6 added "f-strings": `blah = "blah"; eggs = 2; f'spam={blah} eggs={eggs}'`.

PYTHON HAS VARIOUS KINDS OF STRING LITERALS:

Strings delimited by single or double quote marks. Unlike in Unix shells, Perl and Perl-influenced languages, single quote marks and double quote marks function identically. Both kinds of string use the backslash (`\`) as an escape character. String interpolation became available in Python 3.6 as "formatted string literals".

Triple-quoted strings, which begin and end with a series of three single or double quote marks. They may span multiple lines and function like here documents in shells, Perl and Ruby.

Raw string varieties, denoted by prefixing the string literal with an `r`. Escape sequences are not interpreted; hence raw strings are useful where literal backslashes are common, such as regular expressions and Windows-style paths. Compare "@-quoting" in C#.

Python has array index and array slicing expressions on lists, denoted as `a[key]`, `a[start:stop]` or `a[start:stop:step]`. Indexes are zero-based, and negative indexes are relative to the end. Slices take elements from the start index up to, but not including, the stop index. The third slice parameter, called step or stride, allows elements to be skipped and reversed. Slice indexes may be omitted, for example `a[:]` returns a copy of the entire list. Each element of a slice is a shallow copy.

In Python, a distinction between expressions and statements is rigidly enforced, in contrast to languages such as Common Lisp, Scheme, or Ruby. This leads to duplicating some functionality. For example:

List comprehensions vs. for-loops Conditional expressions vs. if blocks

The `eval()` vs. `exec()` built-in functions (in Python 2, `exec` is a statement); the former is for expressions, the latter is for statements.

Statements cannot be a part of an expression, so list and other comprehensions or lambda expressions, all being expressions, cannot contain statements.

METHODS

Methods on objects are functions attached to the object's class; the syntax `instance.method(argument)` is, for normal methods and functions, syntactic sugar for `Class.method(instance, argument)`. Python methods have an explicit `self` parameter to access instance data, in contrast to the implicit `self` (or `this`) in some other object-oriented programming languages.

APPLICATIONS OF PYTHON

As mentioned before, Python is one of the most widely used language over the web. I'm going to list few of them here:

EASY-TO-LEARN – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.

EASY-TO-READ – Python code is more clearly defined and visible to the eyes.

EASY-TO-MAINTAIN – Python's source code is fairly easy-to-maintain.

A BROAD STANDARD LIBRARY – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

INTERACTIVE MODE – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

PORTABLE – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.

EXTENDABLE – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.

DATABASES – Python provides interfaces to all major commercial databases.

GUI PROGRAMMING – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.

SCALABLE – Python provides a better structure and support for large programs than shell scripting.

INSTALLATION STEPS OF PYTHON

Installing and using Python on Windows 10 is very simple. The installation procedure involves just three steps:

- Download the binaries
- Run the Executable installer
- Add Python to PATH environmental variables

To install Python, you need to download the official Python executable installer. Next,

STEP 1: Download the Python

Installer binaries

- Open the official Python website in your web browser. Navigate to the Downloads tab for Windows. Choose the latest Python 3 release. In our example, we choose the latest Python 3.7.3 version. Click on the link to download Windows x86 executable installer if you are using a 32-bit installer.
- In case your Windows installation is a 64-bit system, then download Windows x86-64 executable installer.

STEP 2: Run the Executable Installer

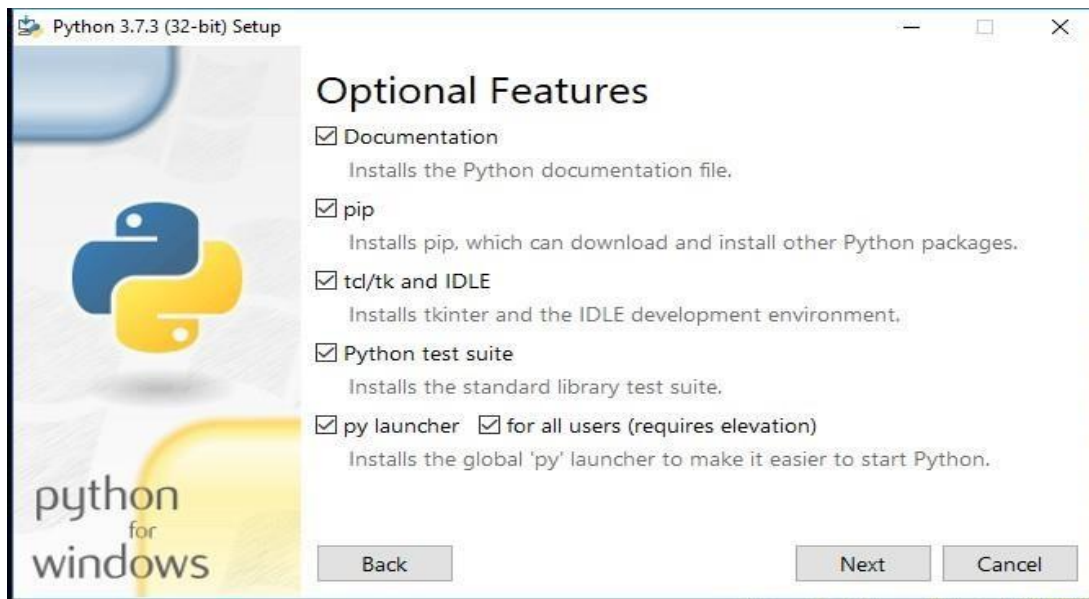
1. Once the installer is downloaded, run the Python installer.
2. Check the Install launcher for all users check box. Further, you may check the Add Python 3.7 to path check box to include the interpreter in the execution path.



3. Select **Customize installation**.

Choose the optional features by checking the following check boxes:

- a. Documentation
 - b. pip
 - c. tcl/tk and IDLE (to install tkinter and IDLE)
 - d. Python test suite (to install the standard library test suite of Python)
 - e. Install the global launcher for `.py` files. This makes it easier to start Python
- Install for all users.

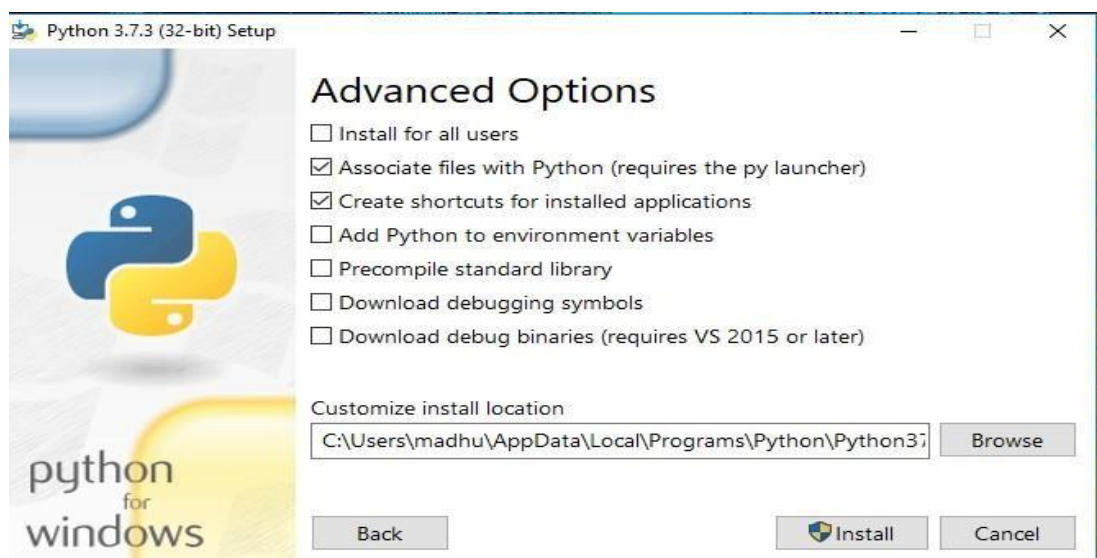


Click Next.

4. This takes you to Advanced Options available while installing Python. Here, select the Install for all users and Add Python to environment variables check boxes.

Optionally, you can select the Associate files with Python, Create shortcuts for installed applications and other advanced options. Make note of the python installation directory displayed in this step. You would need it for the next step.

After selecting the Advanced options, click Install to start installation.



5. Once the installation is over, you will see a Python Setup Successful window.



STEP 3: Add Python to environmental variables

The last (optional) step in the installation process is to add Python Path to the System Environment variables. This step is done to access Python through the command line. In case you have added Python to environment variables while setting the Advanced options during the installation procedure, you can avoid this step. Else, this step is done manually as follows.

In the Start menu, search for “advanced system settings”. Select “View advanced system settings”. In the “System Properties” window, click on the “Advanced” tab and then click on the “Environment Variables” button.

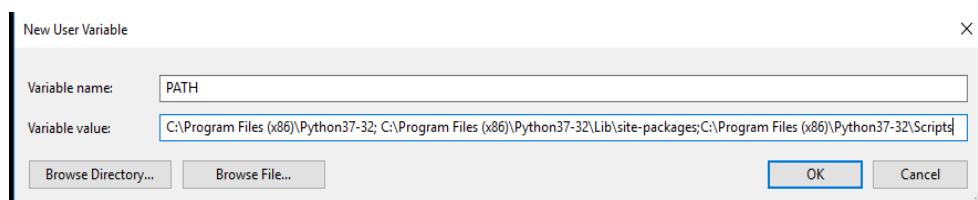
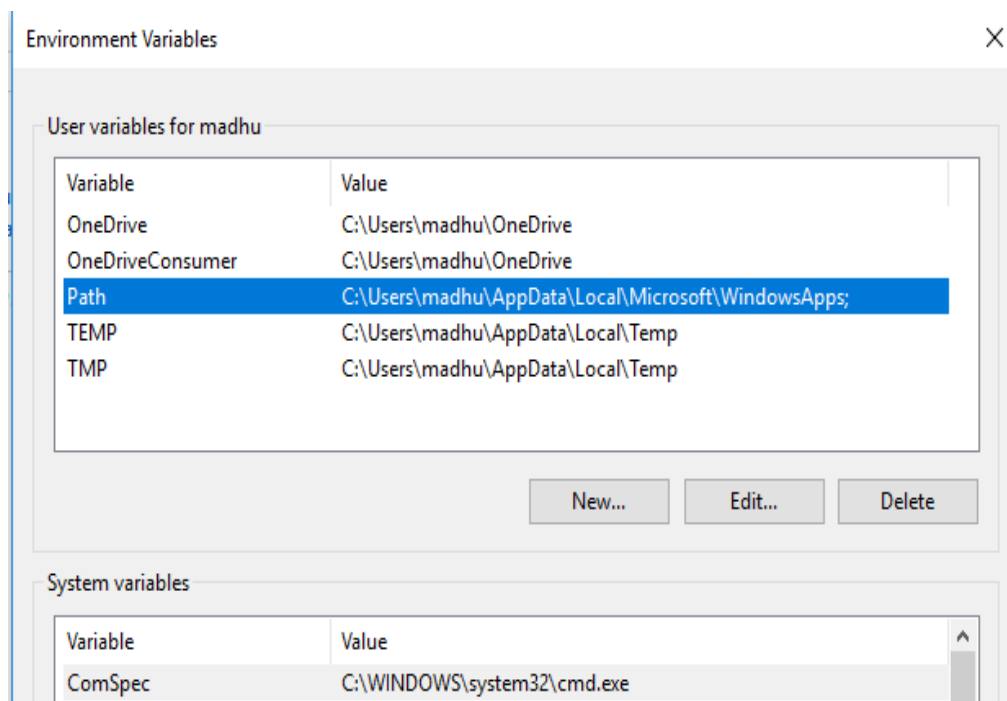
Locate the Python installation directory on your system. If you followed the steps exactly as above, python will be installed in below locations:

- C:\Program Files (x86)\Python37-32: for 32-bit installation
- C:\Program Files\Python37-32: for 64-bit installation.

The folder name may be different from “Python37-32” if you installed a different version.

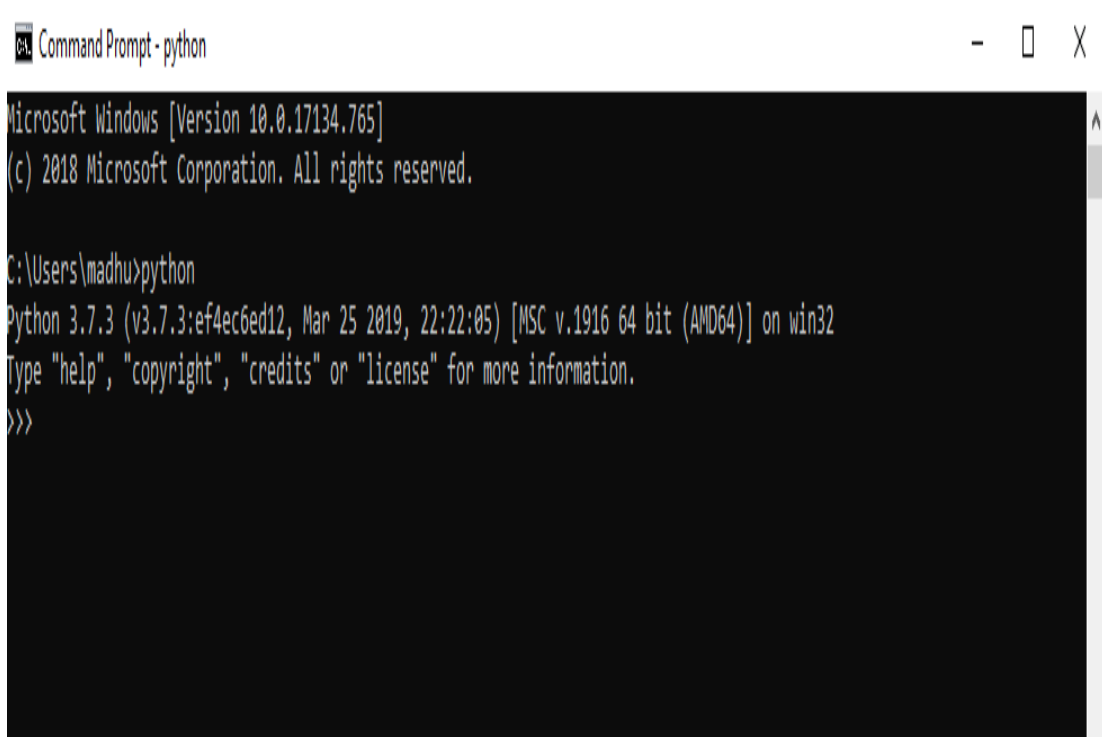
Look for a folder whose name starts with Python.

Append the following entries to PATH variable as shown below



STEP 4: Verify the Python Installation

You have now successfully installed Python 3.7.3 on Windows 10. You can verify if the Python installation is successful either through the command line or through the IDLE app that gets installed along with the installation. Search for the command prompt and type “python”. You can see that Python 3.7.3 is successfully installed.

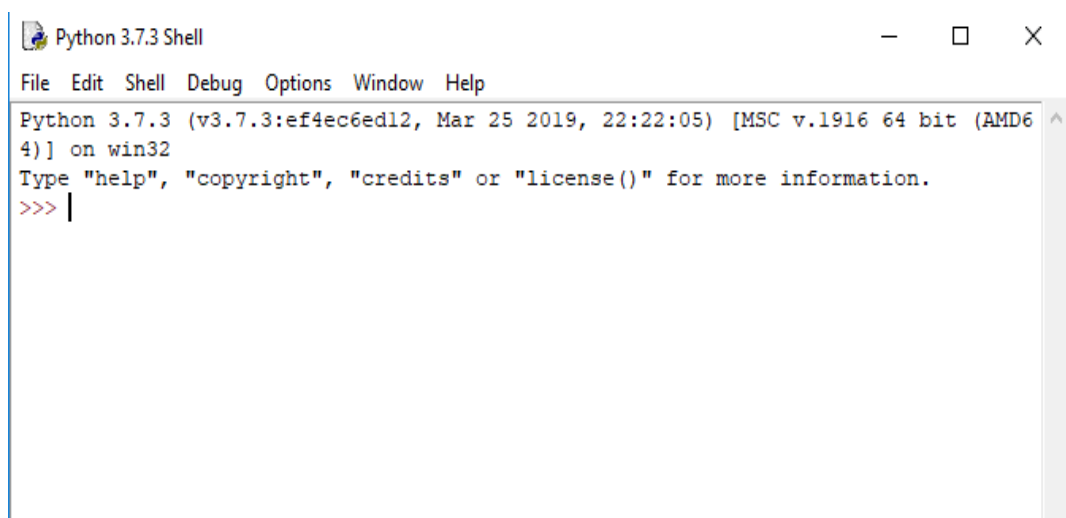


```
Command Prompt - python

Microsoft Windows [Version 10.0.17134.765]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\madhu>python
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 22:22:05) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

An alternate way to reach python is to search for “Python” in the start menu and clicking on IDLE(Python 3.7 64-bit). You can start coding in Python using the Integrated Development Environment(IDLE).



```
Python 3.7.3 Shell

File Edit Shell Debug Options Window Help

Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 22:22:05) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

USES

Since 2003, Python has consistently ranked in the top ten most popular programming languages in the TIOBE Programming Community Index where, as of February 2020, it is the third most popular language (behind Java, and C). It was selected Programming Language of the Year in 2007, 2010, and 2018.

- An empirical study found that scripting languages, such as Python, are more productive than conventional languages, such as C and Java, for programming problems involving string manipulation and search in a dictionary, and determined that memory consumption was often "better than Java and not much worse than C or C++".
- Large organizations that use Python include Wikipedia, Google, Yahoo!, CERN, NASA, Facebook, Amazon, Instagram, Spotify and some smaller entities like ILM and ITA. The social news networking site Reddit is written entirely in Python.
- Python can serve as a scripting language for web applications, e.g., via `mod_wsgi` for the Apache web server. With Web Server Gateway Interface, a standard API has evolved to facilitate these applications. Web frameworks like Django, Pylons, Pyramid, TurboGears, `web2py`, Tornado, Flask, Bottle and Zope support developers in the design and maintenance of complex applications. `Pyjs` and `IronPython` can be used to develop the client-side of Ajax-based applications.
- `SQLAlchemy` can be used as data mapper to a relational database. `Twisted` is a framework to program communications between computers, and is used (for example) by Dropbox.
- Libraries such as `NumPy`, `SciPy` and `Matplotlib` allow the effective use of Python in scientific computing, with specialized libraries such as `Biopython` and `Astropy` providing domain-specific functionality. `SageMath` is a mathematical software with a notebook interface programmable in Python: its library covers many aspects of mathematics including algebra, combinatorics, numerical mathematics, number theory, and calculus.
- Python has been successfully embedded in many software products as a scripting language, including in finite element method software such as Abaqus, 3D parametric modeler like FreeCAD, 3D animation packages such as 3ds Max, Blender, Cinema 4D, Lightwave, Houdini, Maya, modo, MotionBuilder, Softimage, the visual effects
- Compositor Nuke, 2D imaging programs like GIMP, Inkscape, Scribus and Paint Shop Pro, and musical notation programs like scorewriter and capella. GNU Debugger uses

Python as a pretty printer to show complex structures such as C++ containers.

- Python is commonly used in artificial intelligence projects with the help of libraries like TensorFlow, Keras, Pytorch and Scikit-learn. As a scripting language with modular architecture, simple syntax and rich text processing tools, Python is often used for natural language processing.
- Many operating systems include Python as a standard component. It ships with most Linux distributions, AmigaOS 4, FreeBSD (as a package), NetBSD, OpenBSD (as a package) and macOS and can be used from the command line (terminal). Many Linux distributions use installers written in Python: Ubuntu uses the Ubiquity installer, while Red Hat Linux and Fedora use the Anaconda installer. Gentoo Linux uses Python in its package management system, Portage.
- Python is used extensively in the information security industry, including in exploit development.
- Most of the Sugar software for the One Laptop per Child XO, now developed at Sugar Labs, is written in Python. The Raspberry Pi single-board computer project has adopted Python as its main user-programming language.
- Due to Python's user-friendly conventions and easy-to-understand language, it is commonly used as an intro language into computing sciences with students. This allows students to easily learn computing theories and concepts and then apply them to other programming languages.
- LibreOffice includes Python, and intends to replace Java with Python. Its Python ScriptingProvider is a core feature[169] since Version 4.0 from 7 February 2013.
- Python is a general-purpose, popular programming language, and it is used in almost every technical field. The various areas of Python use are given below.
- Data Science: Data Science is a vast field, and Python is an important language for this field because of its simplicity, ease of use, and availability of powerful data analysis and visualization libraries like NumPy, Pandas, and Matplotlib.
- Desktop Applications: PyQt and Tkinter are useful libraries that can be used in GUI - Graphical User Interface-based Desktop Applications. There are better languages for this field, but it can be used with other languages for making Applications.
- Console-based Applications: Python is also commonly used to create command-line or console-based applications because of its ease of use and support for advanced.
- Mobile Applications: While Python is not commonly used for creating mobile applications, it can still be combined with frameworks like Kivy or BeeWare to create cross-platform mobile applications.

- **Software Development:** Python is considered one of the best software-making languages. Python is easily compatible with both from Small Scale to Large Scale software.
- **Artificial Intelligence:** AI is an emerging Technology, and Python is a perfect language for artificial intelligence and machine learning because of the availability of powerful libraries such as TensorFlow, Keras, and PyTorch.
- **Web Applications:** Python is commonly used in web development on the backend with frameworks like Django and Flask and on the front end with tools like JavaScript, HTML, and CSS.
- **Enterprise Applications:** Python can be used to develop large-scale enterprise applications with features such as distributed computing, networking, and parallel processing.
- **3D CAD Applications:** Python can be used for 3D computer-aided design (CAD) applications through libraries such as Blender.
- **Machine Learning:** Python is widely used for machine learning due to its simplicity, ease of use, and availability of powerful machine learning libraries.
- **Computer Vision or Image Processing Applications:** Python can be used for computer vision and image processing applications through powerful libraries such as OpenCV and Scikit-image.
- **Speech Recognition:** Python can be used for speech recognition applications through libraries such as SpeechRecognition and PyAudio.
- **Scientific computing:** Libraries like NumPy, SciPy, and Pandas provide advanced numerical computing capabilities for tasks like data analysis, machine learning, and more.
- **Education:** Python's easy-to-learn syntax and availability of many resources make it an ideal language for teaching programming to beginners.
- **Testing:** Python is used for writing automated tests, providing frameworks like unit tests and pytest that help write test cases and generate reports.
- **Gaming:** Python has libraries like Pygame, which provide a platform for developing games using Python.
- **IoT:** Python is used in IoT for developing scripts and applications for devices like Raspberry Pi, Arduino, and others.
- **Networking:** Python is used in networking for developing scripts and applications for network automation, monitoring, and management.

- DevOps: Python is widely used in DevOps for automation and scripting of infrastructure management, configuration management, and deployment processes.
- Finance: Python has libraries like Pandas, Scikit-learn, and Statsmodels for financial modeling and analysis.
- Audio and Music: Python has libraries like Pyaudio, which is used for audio processing, synthesis, and analysis, and Music21, which is used for music analysis and generation.
- Writing scripts: Python is used for writing utility scripts to automate tasks like file operations, web scraping, and data processing. Python has wide range of libraries and frameworks widely used in various fields such as machine learning, artificial intelligence, web applications, etc. We define some popular frameworks and libraries of Python as follows.
- Web development (Server-side) - Django Flask, Pyramid, CherryPy
- GUIs based applications - Tkinter, PyGTK, PyQt, PyJs, etc.
- Machine Learning - TensorFlow, PyTorch, Scikit-learn, Matplotlib, Scipy, etc.
- Mathematics - NumPy, Pandas, etc.
- BeautifulSoup: a library for web scraping and parsing HTML and XML
- Requests: a library for making HTTP requests
- SQLAlchemy: a library for working with SQL databases
- Kivy: a framework for building multi-touch applications
- Pygame: a library for game development
- Pytest: a testing framework for Python Django
- REST framework: a toolkit for building RESTful APIs
- FastAPI: a modern, fast web framework for building APIs
- Streamlit: a library for building interactive web apps for machine learning and data science
- NLTK: a library for natural language processing

PYTHON FUNCTIONS

- **Lambda Function** - A lambda function is a small, anonymous function that can take any number of arguments but can only have one expression.
- **Map Function** - The `map()` function applies a given function to each item of an iterable and returns a new iterable with the results. The input iterable can be a list, tuple, or other.
- **Filter Function** - The `filter()` function returns an iterator from an iterable for which the function passed as the first argument returns True. It filters out the items from an iterable that do not meet the given condition.
- **Reduce Function** - The `reduce()` function applies a function of two arguments cumulatively to the items of an iterable from left to right to reduce it to a single value.
- **functools Module** - The `functools` module in Python provides higher-order functions that operate on other functions, such as `partial()` and `reduce()`.
- **Currying Function** - A currying function is a function that takes multiple arguments and returns a sequence of functions that each take a single argument.
- **Memoization Function** - Memoization is a technique used in functional programming to cache the results of expensive function calls and return the cached Result when the same inputs occur again.
- **Threading Function** - Threading is a technique used in functional programming to run multiple tasks simultaneously to make the code more efficient and faster.

PYTHON MODULES

- **Math**: Gives users access to mathematical constants and pi and trigonometric functions.
- **Datetime**: Provides classes for a simpler way of manipulating dates, times, and periods.
- **OS**: Enables interaction with the base operating system, including administration of processes and file system activities.
- **Random**: The `random` function offers tools for generating random integers and picking random items from a list.
- **JSON**: JSON is a data structure that can be encoded and decoded and is frequently used in online APIs and data exchange. This module allows dealing with JSON.
- **Re**: Supports regular expressions, a potent text-search and text-manipulation tool.

- Collections: Provides alternative data structures such as sorted dictionaries, default dictionaries, and named tuples.
- NumPy: NumPy is a core toolkit for scientific computing that supports numerical operations on arrays and matrices.

PYTHON FILE I/O

- Files are used to store data in a computer disk. In this tutorial, we explain the built-in file object of Python. We can open a file using Python script and perform various operations such as writing, reading, and appending. There are various ways of opening a file. We are explained with the relevant example. We will also learn to perform read/write operations on binary files.
- Python's file input/output (I/O) system offers programs to communicate with files stored on a disc. Python's built-in methods for the file object let us carry out actions like reading, writing, and adding data to files.
- The open() method in Python makes a file object when working with files. The name of the file to be opened and the mode in which the file is to be opened are the two parameters required by this function. The mode can be used according to work that needs to be done with the file, such as "r" for reading, "w" for writing, or "a" for attaching.
- After successfully creating an object, different methods can be used according to our work. If we want to write in the file, we can use the write() functions, and if you want to read and write both, then we can use the append() function and, in cases where we only want to read the content of the file we can use read() function. Binary files containing data in a binary rather than a text format may also be worked with using Python. Binary files are written in a manner that humans cannot directly understand. The rb and wb modes can read and write binary data in binary files

DJANGO FRAMEWORK

- Django is a web application framework written in Python programming language. It is based on MVT (Model View Template) design pattern. The Django is very demanding due to its rapid development feature. It takes less time to build application after collecting client requirement. This framework uses a famous tag line: The web framework for perfectionists with deadlines.
- By using Django, we can build web applications in very less time.

- Django is designed in such a manner that it handles much of configure things automatically, so we can focus on application development only.
- Python Django is a web framework that allows to quickly create efficient web pages. Django is also called batteries included framework because it provides built-in features.
- Django is based on MVT (Model-View-Template) architecture which has the following
 - **Model:** The model is going to act as the interface of your data. It is responsible for maintaining data. It is the logical data structure behind the entire application and is represented by a database (generally relational databases such as MySQL, Postgres).
 - **View:** The View is the user interface that you see in your browser when you render a website. It is represented by HTML/CSS/Javascript and Jinja files.
 - **Template:** A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted. To check more, visit – Django Templates.

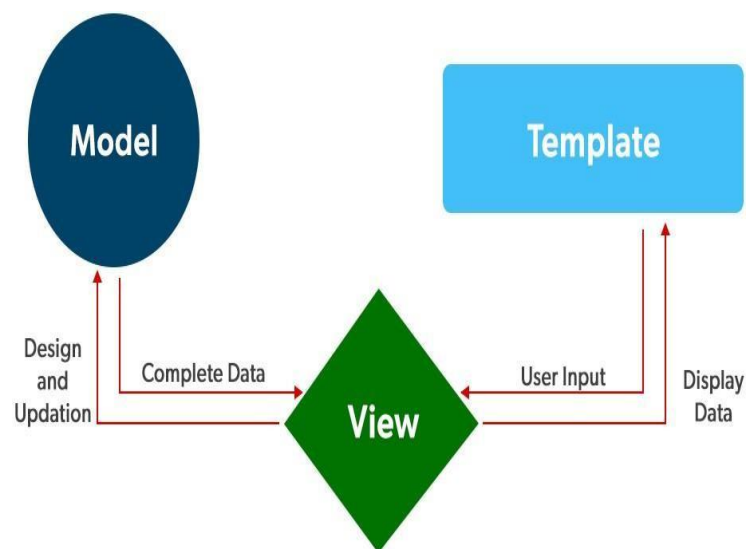


Fig: Django Framework

8. SYSTEM TESTING

Software Testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding, Testing presents an interesting anomaly for the software engineer

TESTING OBJECTIVES INCLUDE:

- a. Testing is a process of executing a program with the intent of finding an error
- b. A good test case is one that has a probability of finding an as yet undiscovered error
- c. A successful test is one that uncovers an undiscovered error

TESTING PRINCIPLES:

- All tests should be traceable to end user requirements
- Tests should be planned long before testing begins
- Testing should begin on a small scale and progress towards testing in large
- Exhaustive testing is not possible
- To be most effective testing should be conducted by a independent third party

TESTING STRATEGIES

A Strategy for software testing integrates software test cases into a series of well planned steps that result in the successful construction of software. Software testing is a broader topic for what is referred to as Verification and Validation. Verification refers to the set of activities that ensure that the software correctly implements a specific function Validation refers to the set of activities that ensure that the software that has been built is traceable to customer's requirements.

UNIT TESTING:

Unit testing focuses verification effort on the smallest unit of software design that is the module.

INTEGRATION TESTING:

Integration testing is a systematic technique for constructing the program structure while conducting test to uncover errors associated with the interfacing. The objective is to take unit tested methods and build a program structure that has been dictated by design.

TOP-DOWN INTEGRATION:

Top down integrations an incremental approach to construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main control program. Modules subordinate to the main program are incorporated in the structure either in the breath-first or depth-first manner.

BOTTOM-UP INTEGRATION:

This method as the name suggests, begins construction and testing with atomic modules i.e., modules at the lowest leveling the program structure. Because the modules are integrated in the bottom up manner the processing required for the modules subordinate to a given level is always available and the need for stubs is eliminated.

VALIDATION TESTING:

At the end of integration testing software is completely assembled as a package. Validation testing is the next stage which can be defined as successful when the software functions in the manner reasonably expected by the customer. Reasonable expectations are those defined in the software requirements specifications. Information contained in those sections form a basis for validation testing approach.

SYSTEM TESTING:

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated to perform allocated function.

TEST CASES:

TC. No	Test Case	Input	Expected Output	Observed Output	Result
1	Login	Enter Wrong User Name and Password	Invalid Login Details	User name and Password are invalid	Pass
2	Login	Enter User Name and Password	Login Successful	Login Successful	Pass
3	Mobile Number	Enter Alphanumeric characters	Mobile number must be digits only	Mobile number in 10 digits only	Fail
4	Upload file	Browse file	file uploaded successfully	file Uploaded successful	Pass

9. SCREEN SHOTS

STRAWBERRY SHELF LIFE HOME PAGE

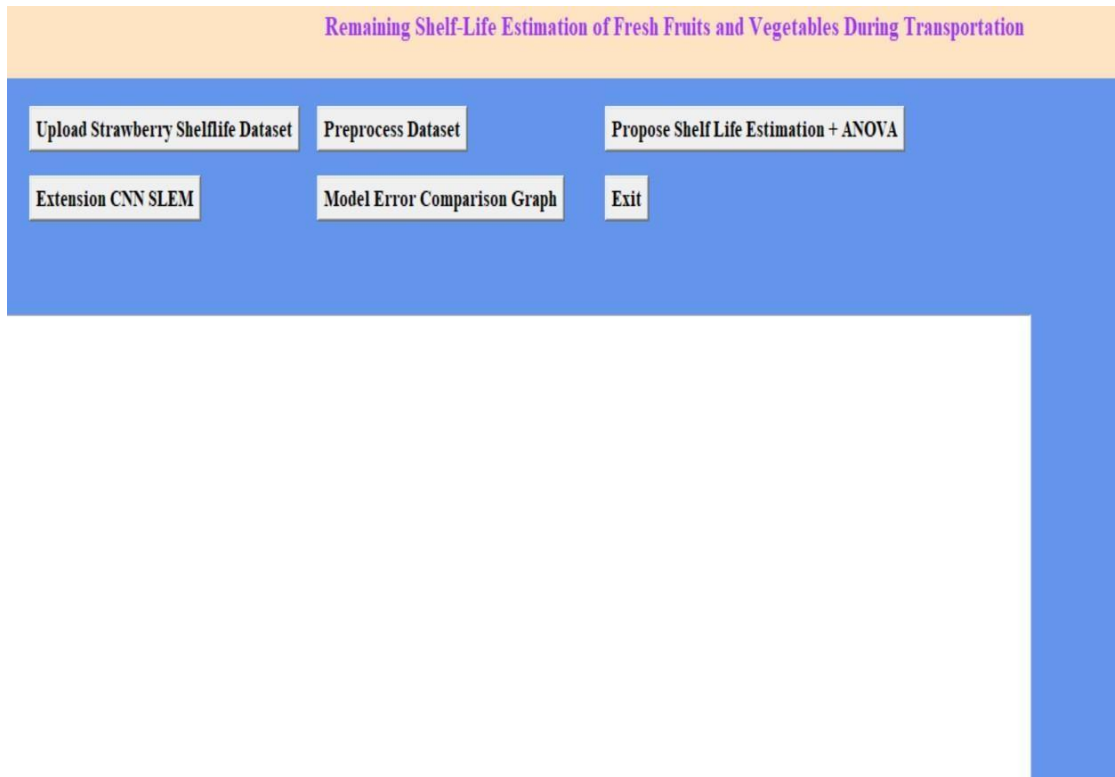


Fig: Strawberry Shelf life Home page

DOOR OPENING-CLEANED

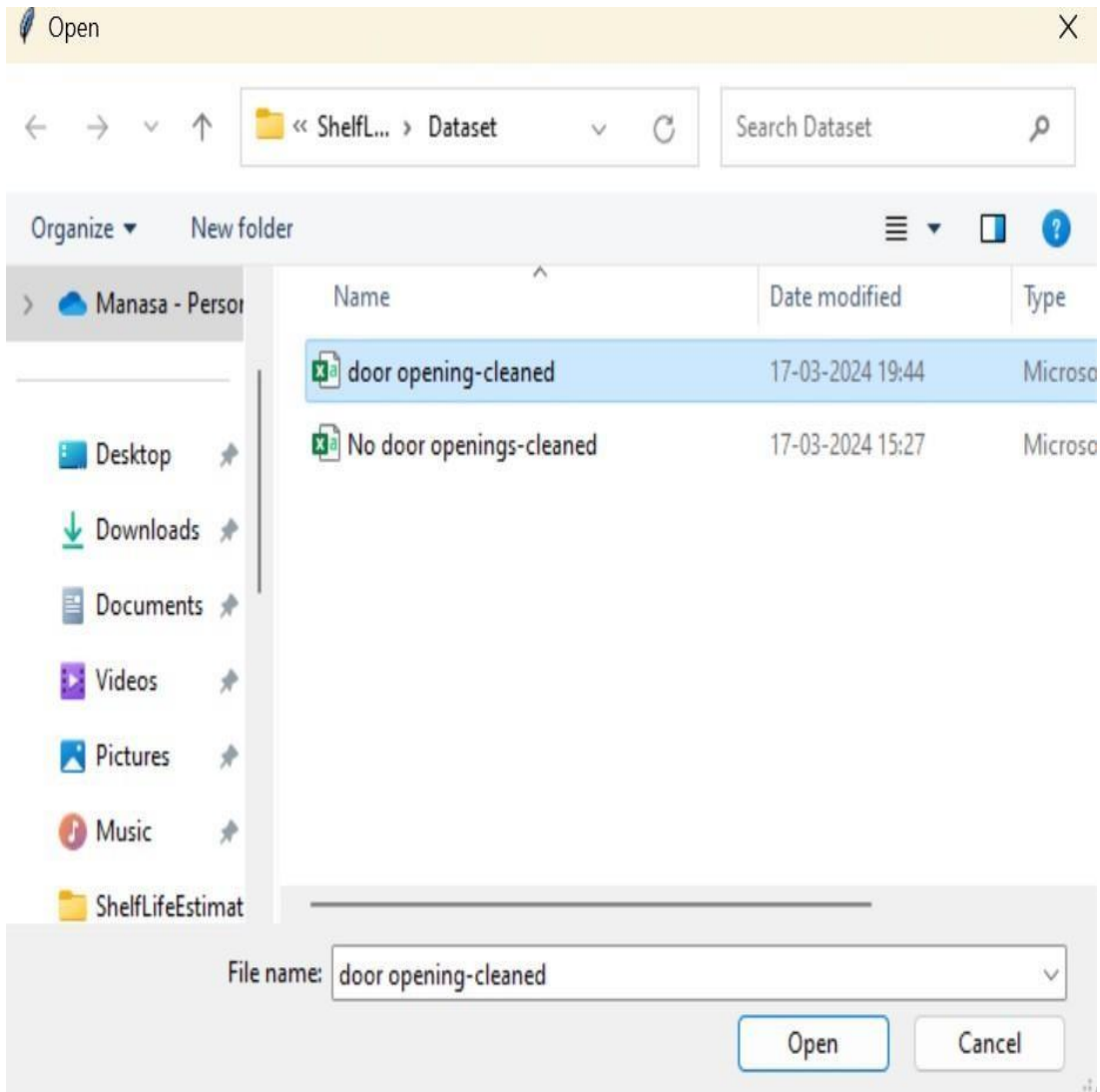


Fig: Door Opening-Cleaned

UPLOAD STRAWBERRY SHELF LIFE DATA

C:/Users/manas/Desktop/ShelfLifeEstimation/Dataset/door opening-cleaned.csv loaded

	Time	Temp
0	0.0	-2.2
1	3.0	1.0
2	5.0	1.0
3	6.0	5.6
4	6.0	5.6

Fig: Upload Strawberry Shelf Life Data Set

PREPROCESS DATA SET

Dataset Processing RSL (Remaining Shelf Life) Calculation Completed

RSL = 0 0.0

1 3.2

2 0.0

3 4.6

4 0.0

...

231 0.0

232 0.0

233 0.0

234 0.0

235 0.0

Name: Diff, Length: 236, dtype: float64

Fig: Preprocess Data Set

PROPOSE SHELF LIFE ESTIMATION + ANOVA GRAPH

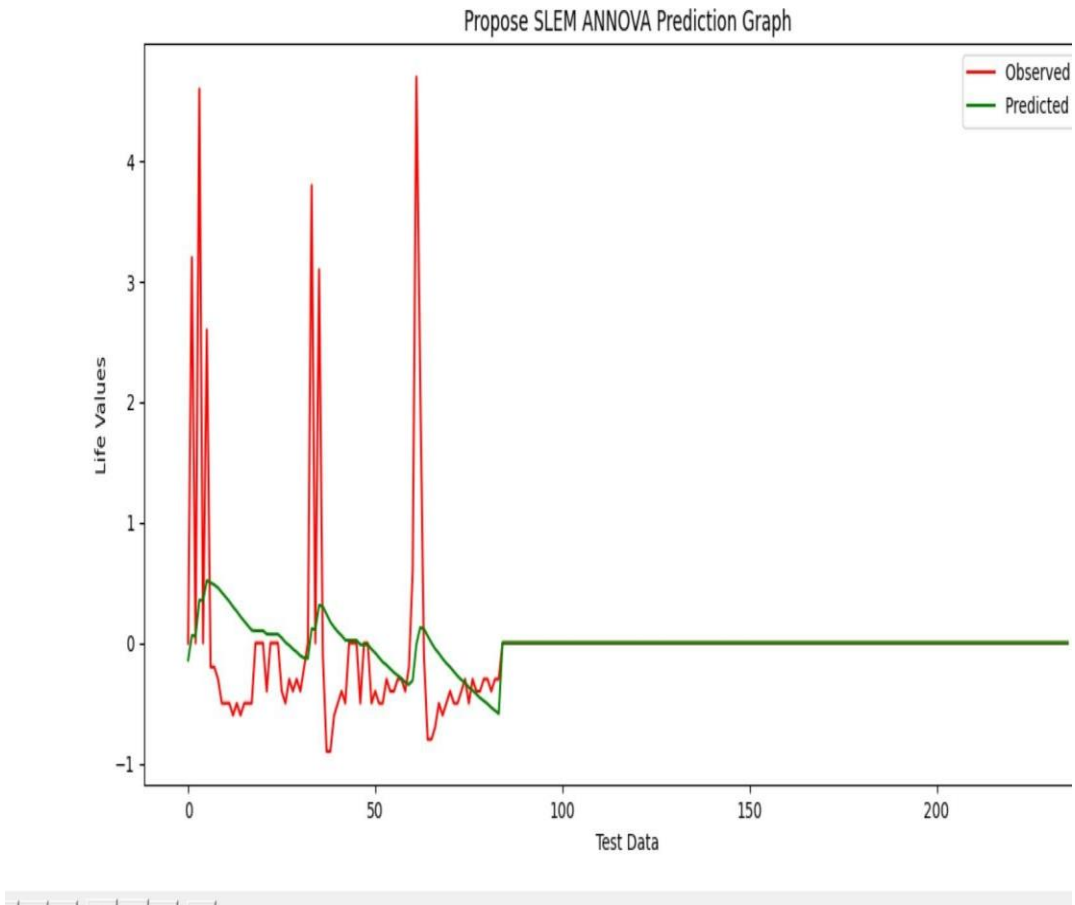


Fig: Propose Shelf Life Estimation + ANOVA Graph

PROPOSE SHELF LIFE ESTIMATION + ANOVA RESULT

Propose SLEM Model Error : 0.6408938280476446

Observed Life : 0 Days Predicted Life : 4 Days
Observed Life : 9 Days Predicted Life : 1 Days
Observed Life : 0 Days Predicted Life : 1 Days
Observed Life : 13 Days Predicted Life : 10 Days
Observed Life : 0 Days Predicted Life : 10 Days
Observed Life : 7 Days Predicted Life : 15 Days
Observed Life : 5 Days Predicted Life : 15 Days
Observed Life : 6 Days Predicted Life : 14 Days
Observed Life : 8 Days Predicted Life : 13 Days
Observed Life : 15 Days Predicted Life : 12 Days

Fig: Propose Shelf Life Estimation + ANOVA Result

EXTENTION CNN SLEM GRAPH

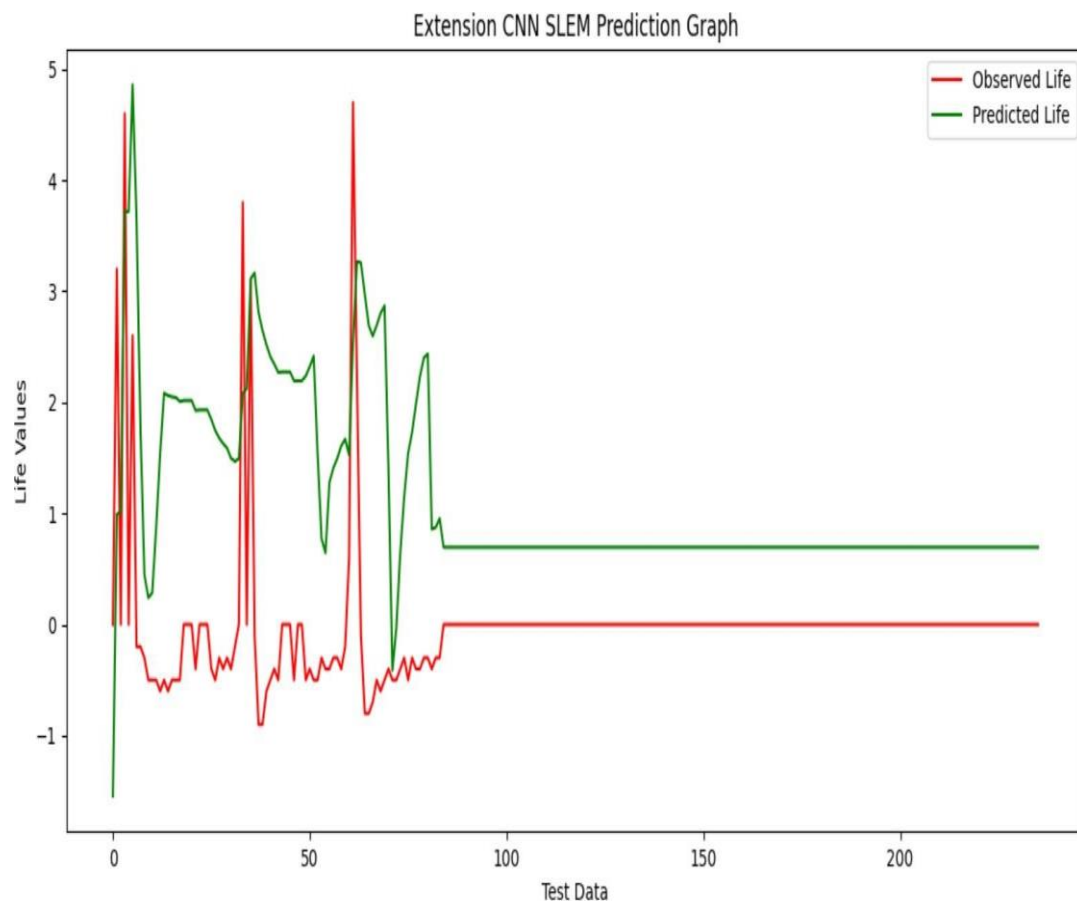


Fig: Extension CNN SLEM Graph

EXTENSION CNN SLEM

Extension CNN Model Error : 0.2630454491532315

Observed Life : 0 Days Predicted Life : 46 Days
Observed Life : 9 Days Predicted Life : 29 Days
Observed Life : 0 Days Predicted Life : 30 Days
Observed Life : 13 Days Predicted Life : 11 Days
Observed Life : 0 Days Predicted Life : 11 Days
Observed Life : 7 Days Predicted Life : 14 Days
Observed Life : 5 Days Predicted Life : 10 Days
Observed Life : 6 Days Predicted Life : 5 Days
Observed Life : 8 Days Predicted Life : 13 Days
Observed Life : 15 Days Predicted Life : 7 Days

Fig: Extension CNN SLEM

MODEL ERROR COMPARISION GRAPH

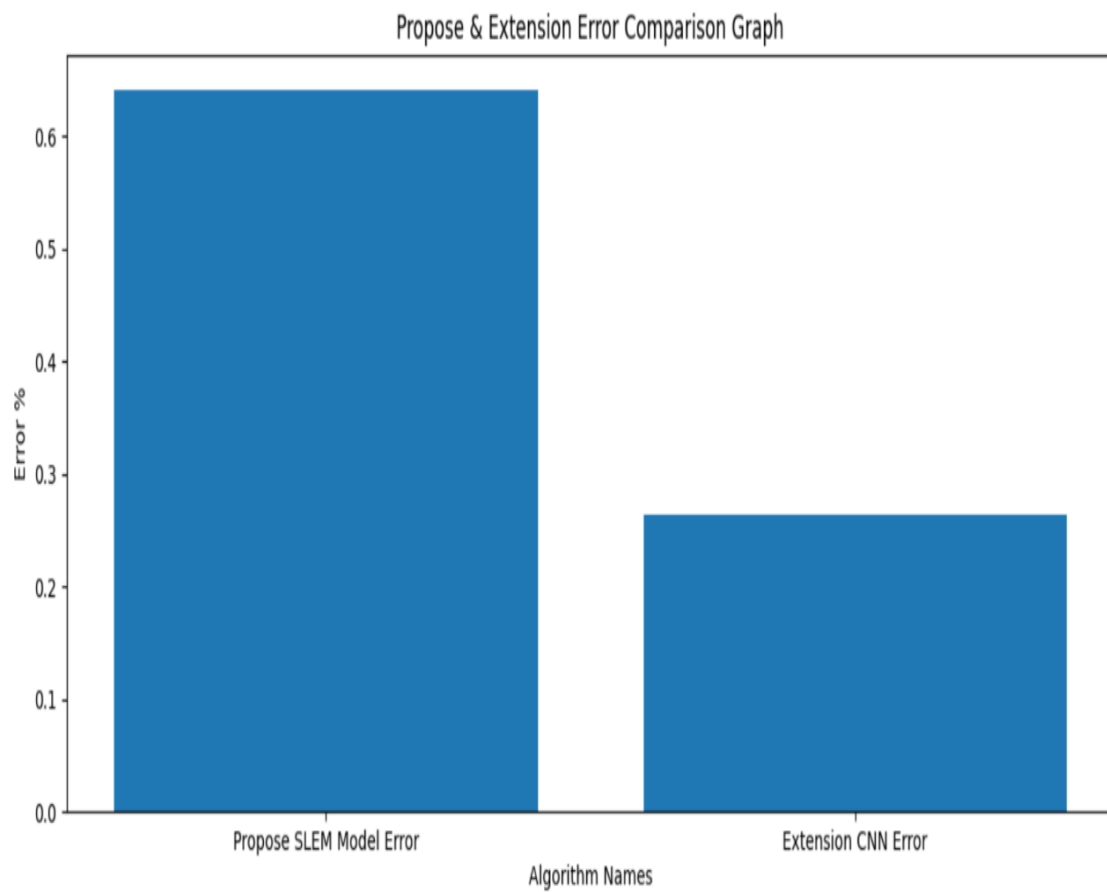


Fig: Model Error Comparison Graph

NO DOOR OPENING-CLEANED

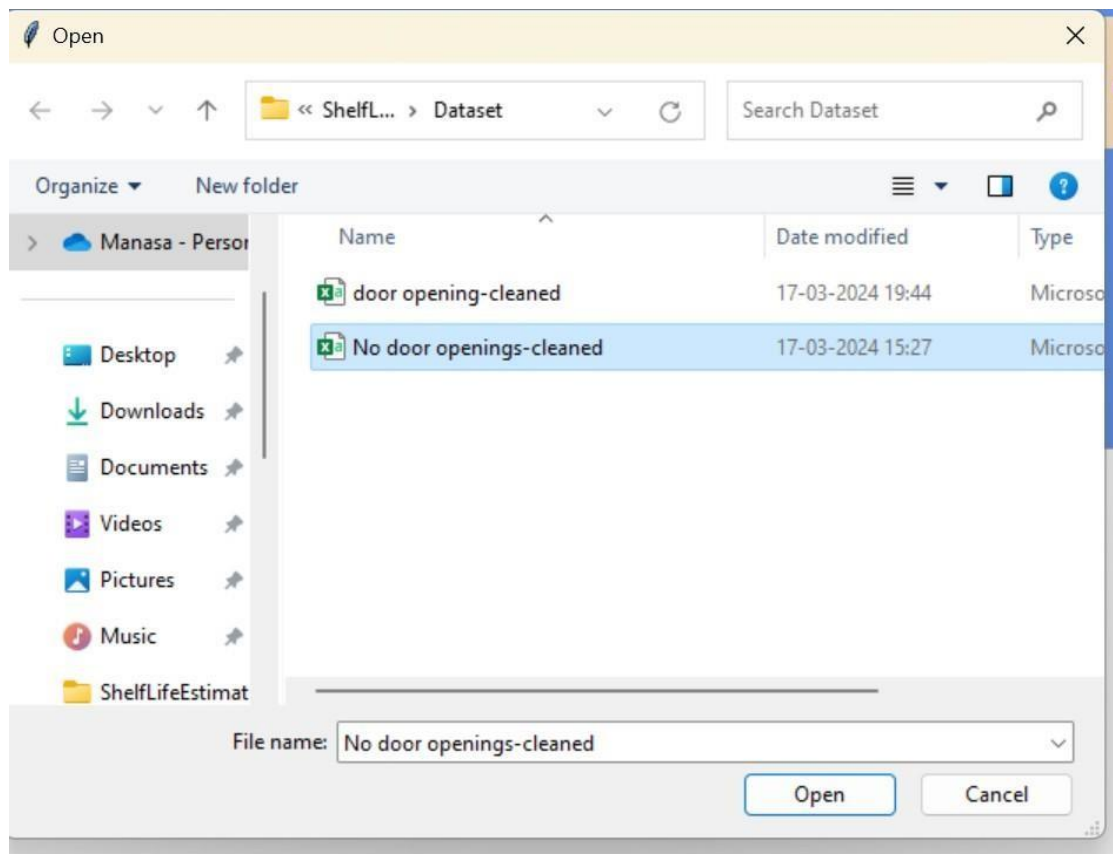
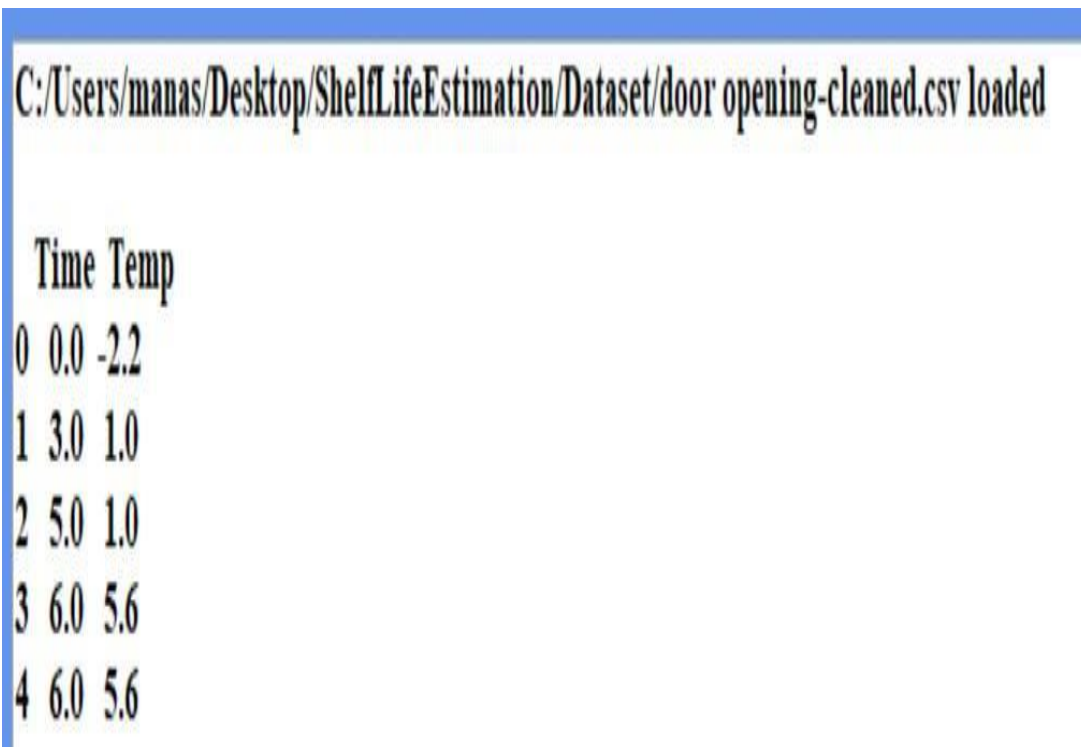


Fig: No Door Opening-Cleaned

UPLOAD STRAWBERRY SHELF LIFE DATA SET



C:/Users/manas/Desktop/ShelfLifeEstimation/Dataset/door opening-cleaned.csv loaded

	Time	Temp
0	0.0	-2.2
1	3.0	1.0
2	5.0	1.0
3	6.0	5.6
4	6.0	5.6

Fig: Upload Strawberry Shelf-Life Data Set

PREPROCESS DATA SET

Dataset Processing RSL (Remaining Shelf Life) Calculation Completed

RSL = 0 0.0

1 -0.1

2 0.3

3 0.4

4 0.3

...

65 -0.4

66 -0.4

67 -0.4

68 -0.1

69 0.2

Name: Diff, Length: 70, dtype: float64

Fig: Preprocess Data Set

PROPOSE SHELF LIFE ESTIMATION + ANOVA GRAPH

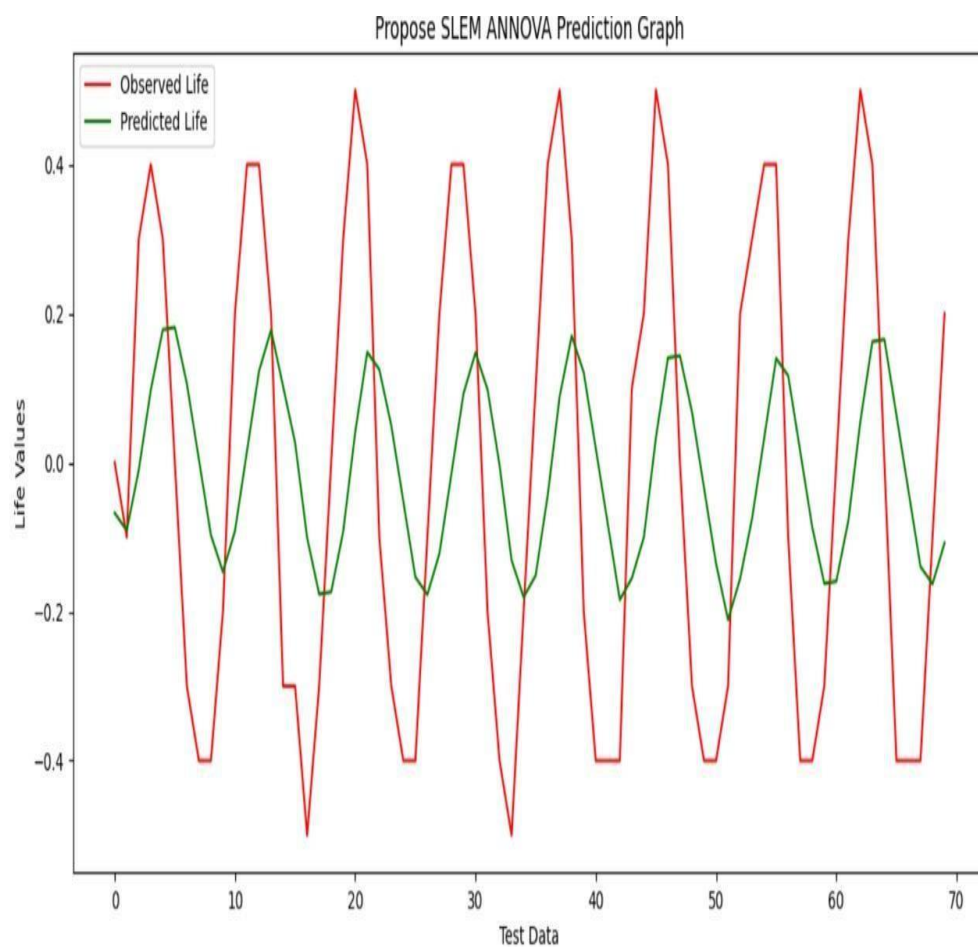


Fig: Propose Shelf Life Estimation + ANOVA Graph

PROPOSE SHELF LIFE ESTIMATION + ANOVA RESULT

Propose SLEM Model Error : 0.3033021086419326

Observed Life : 0 Days Predicted Life : 2 Days

Observed Life : 2 Days Predicted Life : 2 Days

Observed Life : 9 Days Predicted Life : 0 Days

Observed Life : 11 Days Predicted Life : 2 Days

Observed Life : 9 Days Predicted Life : 5 Days

Observed Life : 0 Days Predicted Life : 5 Days

Observed Life : 9 Days Predicted Life : 3 Days

Observed Life : 11 Days Predicted Life : 0 Days

Observed Life : 12 Days Predicted Life : 2 Days

Observed Life : 5 Days Predicted Life : 4 Days

Fig: Propose Shelf Life Estimation + ANOVA Result

EXTENTION CNN SLEM RESULT

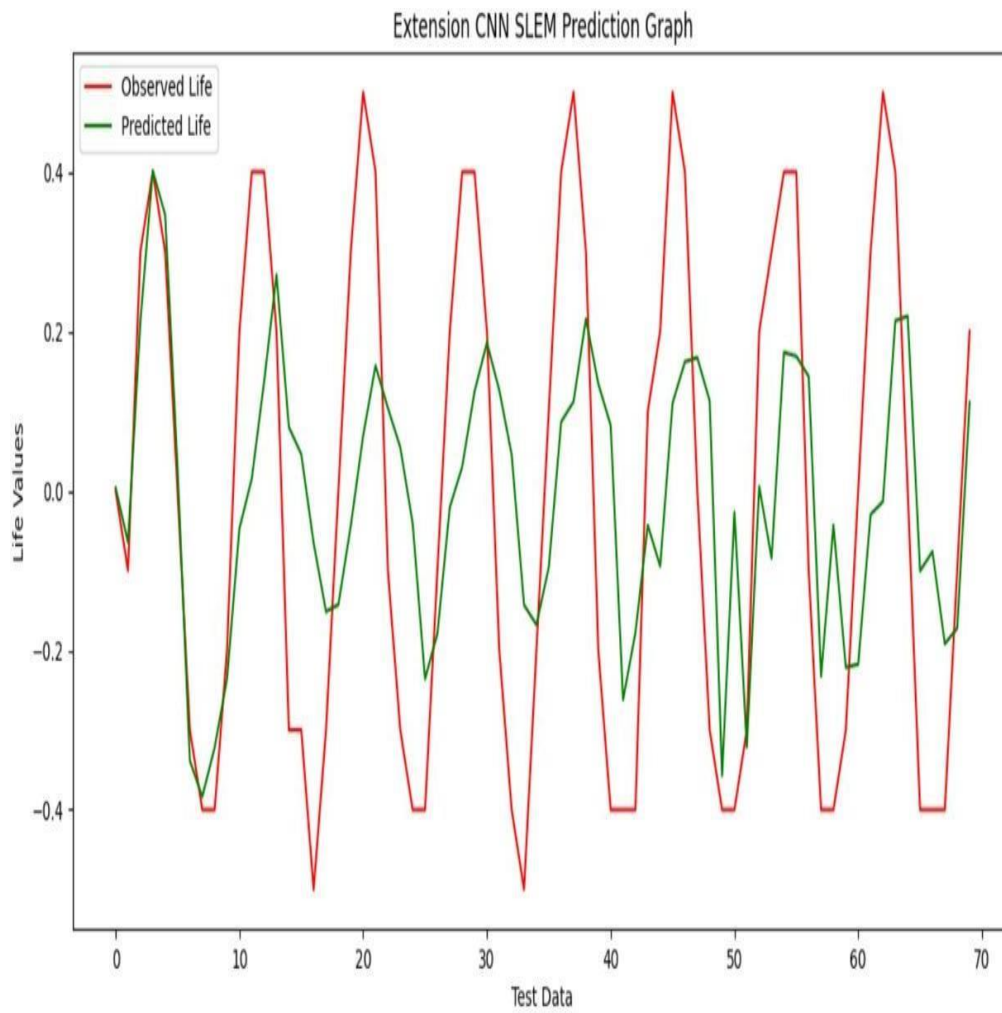


Fig: Extension CNN SLEM Graph

EXTENTION CNN SLEM RESULT

Extension CNN Model Error : 0.2622780837723077

Observed Life : 0 Days Predicted Life : 0 Days
Observed Life : 3 Days Predicted Life : 1 Days
Observed Life : 9 Days Predicted Life : 6 Days
Observed Life : 11 Days Predicted Life : 12 Days
Observed Life : 9 Days Predicted Life : 10 Days
Observed Life : 0 Days Predicted Life : 0 Days
Observed Life : 9 Days Predicted Life : 10 Days
Observed Life : 11 Days Predicted Life : 11 Days
Observed Life : 12 Days Predicted Life : 9 Days
Observed Life : 5 Days Predicted Life : 7 Days

Fig: Extention CNN SLEM Result

MODEL ERROR COMPARISON GRAPH

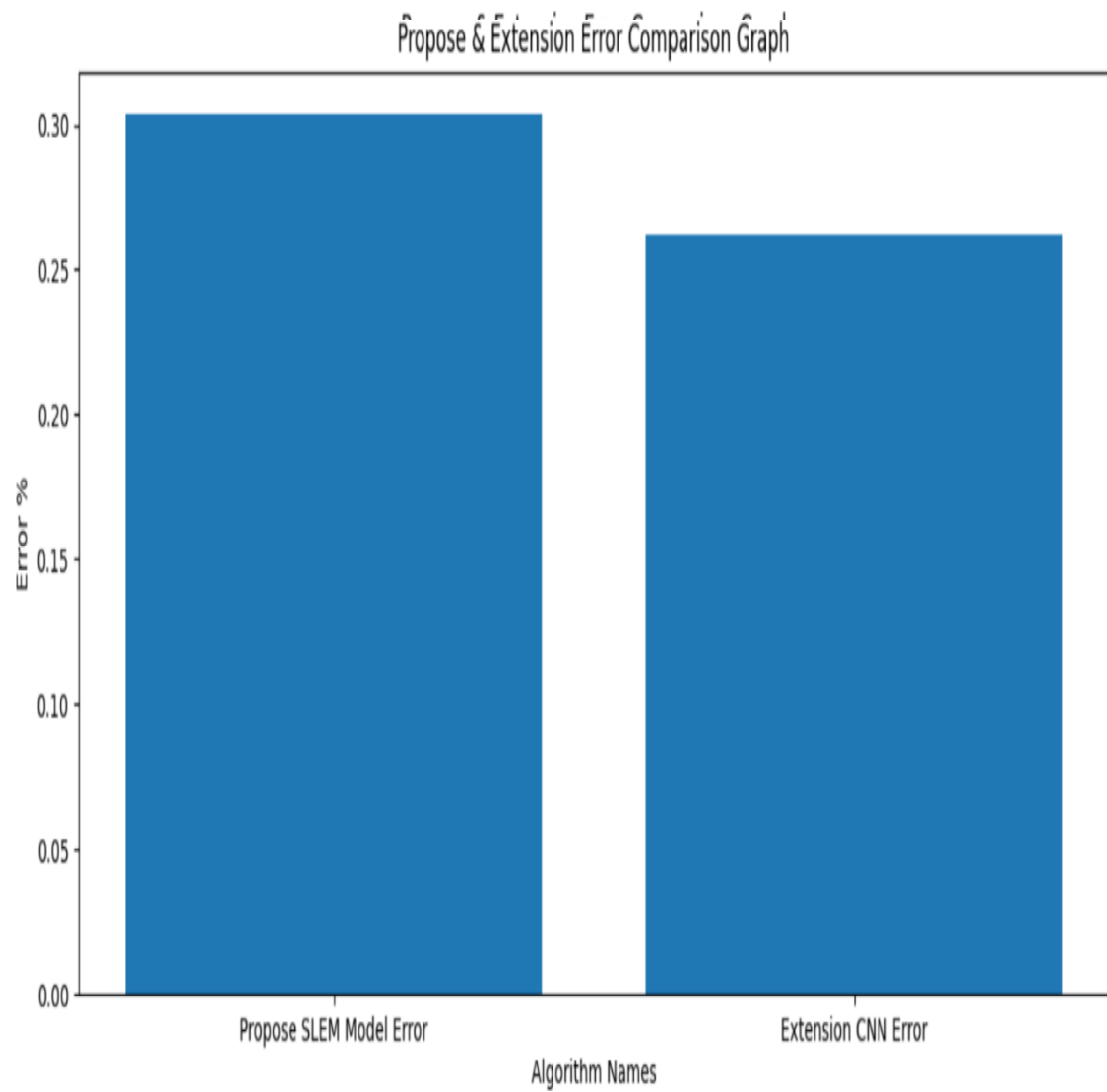


Fig: Model Error Comparison Graph

10. CONCLUSION

This project presented a white-box Shelf-Life Estimation Model (SLEM) implemented in Matlab, which uses any ambient temperature history to estimate the RSL of an FFV in real time. The proposed SLEM was validated experimentally for three fresh products in sealed and unsealed packaging under dynamic temperature profiles. The model performed well with unsealed strawberries and apricots, with errors ranging from 0.04 to 1.2 days in error. The model was exceptionally superior for strawberries, an exemplary nonclimacteric fruit. A surrogate method (QnD) was proposed for computing the pulp temperature for fast, real-time applications, which also produced acceptable results for strawberries and apricots, but performed poorly for bunched and sealed spinach. Despite simplifying assumptions, the SLEM was found to generate statistically sound predictions nonetheless, which supports the view that the respiration rate and its interaction with time, temperature, and the local CO₂ concentration are the key predictors of decay, notwithstanding the effects of microbiological activity, relative humidity, and exogenous ethylene. Even though the original and surrogate model results are statistically sound, the surrogate model is not recommended for sealed fresh products or prolonged experiments if prediction errors exceeding 2 days are unacceptable by the end customer.

FUTURE ENHANCEMENT

In future works, we will expand the diversity of experiments in terms of devices, subjects, and environment conditions to further improve our Shelf-Life Estimation of Fresh Fruits and Vegetables During Transportation.

11. REFERENCES

- [1] S. K. Srivastava, A. Chaudhuri, and R. K. Srivastava, "Propagation of risks and their impact on performance in fresh food retail," *Int. J. Logistics Manag.*, vol. 26, no. 3, pp. 568–602, Nov. 2015.
- [2] J. Gustavsson, C. Cederberg, U. Sonesson, R. Van Otterdijk, and A. Meybeck, "Global food losses and food waste," presented at the Save Food Congr., Düsseldorf, Germany, May 2011. Accessed: Jan. 6, 2022. [Online]. Available: https://www.madr.ro/docs/ind-alimentara/risipa_alimentara/presentation_food_waste.pdf
- [3] Y. Bouzembrak, M. Kluche, A. Gavai, and H. J. P. Marvin, "Internet of Things in food safety: Literature review and a bibliometric analysis," *Trends Food Sci. Technol.*, vol. 94, pp. 54–64, Dec. 2019. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0924224419303048>
- [4] M. Awad, M. Ndiaye, and A. Osman, "Vehicle routing in cold food supply chain logistics: A literature review," *Int. J. Logistics Manag.*, vol. 32, no. 2, pp. 592–617, Apr. 2021, doi: 10.1108/IJLM-02-2020-0092.
- [5] G. Hough, *Sensory Shelf Life Estimation of Food Products*. Boca Raton, FL, USA: CRC Press, May 2010. [Online]. Available: <https://www.taylorfrancis.com/books/9780429147180>
- [6] S. Jol, A. Kassianenko, K. Wszol, and J. Oggel, "Issues in time and temperature abuse of refrigerated foods," *Food Saf.*, vol. 11, no. 6, pp. 30–35, 2006.
- [7] M. L. Hertog, I. Uysal, U. McCarthy, B. M. Verlinden, and B. M. Nicolai, "Shelf life modelling for first-expired-first-out warehouse management," *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 372, no. 2017, Jun. 2014, Art. no. 20130306. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4006170/>
- [8] M. G. Corradini and M. Peleg, "Shelf-life estimation from accelerated storage data," *Trends Food Sci. Technol.*, vol. 18, no. 1, pp. 37–47, Jan. 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0924224406002421>
- [9] M. Peleg and M. G. Corradini, "Microbial growth curves: What the models tell us and what they cannot," *Crit. Rev. Food Sci. Nutrition*, vol. 51, no. 10, pp. 917–945, Dec. 2011.
- [10] C. S. Barsa, M. D. Normand, and M. Peleg, "On models of the temperature effect on the rate of chemical reactions and biological processes in foods," *Food Eng. Rev.*, vol. 4, no. 4, pp. 191–202, Dec. 2012, doi: 10.1007/s12393-012-9056-x.

- [11] M. L. Amodio, A. Derossi, L. Mastrandrea, and G. Colelli, “A study of the estimated shelf life of fresh rocket using a non-linear model,” *J. Food Eng.*, vol. 150, pp. 19–28, Apr.
- [12] C. Watkins and J. Nock, “Production guide for storage of organic fruits and vegetables,” Dept. Horticulture, Cornell University, New York, NY, USA, Tech. Rep., 10, 2012. [Online]. Available: https://plantpathology.ces.ncsu.edu/wp-content/uploads/2013/12/stored_fruit_veg.pdf?pwd=no
- [13] M. Sohail, D.-W. Sun, and Z. Zhu, “Recent developments in intelligent packaging for enhancing food quality and safety,” *Crit. Rev. Food Sci. Nutrition*, vol. 58, no. 15, pp. 2650–2662, Oct. 2018. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/10408398.2018.1449731>
- [14] G. La Scalia, A. Nasca, O. Corona, L. Settanni, and R. Micale, “An innovative shelf life model based on smart logistic unit for an efficient management of the perishable food supply chain,” *J. Food Process Eng.*, vol. 40, no. 1, Feb. 2017, Art. no. e12311. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/jfpe.12311>
- [15] G. La Scalia, L. Settanni, R. Micale, and M. Enea, “Predictive shelf life model based on RF technology for improving the management of food supply chain: A case study,” *Int. J. RF Technol.*, vol. 7, no. 1, pp. 31–42, Jan. 2016. [Online]. Available: <https://www.medra.org/servelet/aliasResolver?alias=iospress&doi=10.3233/RFT-150073>
- [16] R. Sciortino, R. Micale, M. Enea, and G. La Scalia, “A webGISbased system for real time shelf life prediction,” *Comput. Electron. Agricult.*, vol. 127, pp. 451–459, Sep. 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0168169916304719>
- [17] K. Tano, A. Kamenan, and J. Arul, “Respiration and transpiration characteristics of selected fresh fruits and vegetables,” *Agronomie Africaine*, vol. 17, no. 2, pp. 103–115, Feb. 2009. [Online]. Available: <https://www.ajol.info/index.php/aga/article/view/1662>
- [18] K. C. Gross, C. Y. Wang, and M. Saltveit, *The Commercial Storage of Fruits, Vegetables, and Florist and Nursery Stock (Agriculture Handbook)*. Washington, DC, USA: U.S. Department of Agriculture, 2016. [Online]. Available: <http://www.ars.usda.gov/is/np/indexpubs>
- [19] ASHRAE Handbook—Refrigeration, American Society of Heating Refrigerating and Air-Conditioning Engineers (ASHRAE), Atlanta, GA, USA, 2018

- [20] Controlled Atmosphere. Accessed: Sep. 30, 2020. [Online]. Available: <https://www.blueatmosphere.nl/controlled-atmosphere/>
- [21] D. A. Castellanos, J. P. Cerisuelo, P. Hernandez-Muñoz, A. O. Herrera, and R. Gavara, “Modelling the evolution of O₂ and CO₂ concentrations in MAP of a fresh product: Application to tomato,” *J. Food Eng.*, vol. 168, pp. 84–95, Jan. 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0260877415003283>
- [22] B. Wu, Q. Guo, G.-X. Wang, X.-Y. Peng, J.-D. Wang, and F.-B. Che, “Effects of different postharvest treatments on the physiology and quality of ‘Xiaobai’ apricots at room temperature,” *J. Food Sci. Technol.*, vol. 52, no. 4, pp. 2247–2255, Apr. 2015. [Online]. Available: <http://link.springer.com/10.1007/s13197-014-1288-8>
- [23] K. Wohl and W. O. James, “The energy changes associated with plant respiration,” *New Phytologist*, vol. 41, no. 4, pp. 230–256, Dec. 1942.
- [24] D. S. Smith, A. Alzina, J. Bourret, B. Nait-Ali, F. Pennec, N. Tessier-Doyen, K. Otsu, H. Matsubara, P. Elser, and U. T. Gonzenbach, “Thermal conductivity of porous materials,” *J. Mater. Res.*, vol. 28, no. 17, pp. 2260–2272, Sep. 2013.
- [25] A.H. Raval, S. C. Solanki, and R. Yadav, “A simplified heat transfer model for predicting temperature change inside food package kept in cold room,” *J. Food Sci. Technol.*, vol. 50, no. 2, pp. 257–265, Apr. 2013, doi: 10.1007/s13197-011-0342-z.

TERM PAPER

ESTIMATION OF SHELF LIFE OF FRESH FRUITS AND VEGETABLES USING MACHINE LEARNING

¹CH. RUSHYENDRA MANI, ²M. MANASA

¹Asst. Professor, Dept. of MCA, Audisankara College of Engineering and Technology
(AUTONOMOUS) Gudur, Andhra Pradesh, India.

²PG Scholar, Dept. of MCA, Audisankara College of Engineering and
Technology(AUTONOMOUS) Gudur, Andhra Pradesh, India.

ABSTRACT— During transportation, prediction of the Remaining Shelf-Life (RSL) of Fresh Fruits and Vegetables (FFVs) is critical for planning and quality cost estimation. The Internet of Things (IoT) enables measured environmental variables to be processed in real-time. However, there is a need for a validated, real-time computational method that translates environmental measurements to dynamic RSL estimates. Most existing generic RSL models for FFVs are qualitative, invasive, or static. This study establishes a generic RSL model for FFVs under dynamic and unplanned logistic conditions. The model is based on estimating the current rate of general decay based on the expected respiration rate of the product, and integrating the decay rate with respect to time. Its implementation is non-destructive, non-invasive, and does not require accelerated shelf-life experiments before

addition, since the original model is rather computationally intensive, a surrogate model was proposed to allow the model to be implemented in fast, real-time applications for ‘Edge IoT.’ Experimental validation of the model using three fresh products (strawberries, apricots, and spinach) in a domestic refrigerator resulted in a maximum deviation of 1.3 days in prediction error using the original model and 2.95 days using the surrogate model. Nonetheless, the predictions made using either the original or surrogate models were statistically sound and not significantly different from the observed shelf lives of the samples, even at the 0.01 significance level

Index Terms— Remaining shelf life, fresh fruits and vegetables, IoT, food supply chain, quality monitoring, transportation.

I. INTRODUCTION

The waste of Fresh Fruits and Vegetables (FFVs) is a significant issue in food supply chain industries. Srivastava et al. reported that food retailers experience profit margin losses due to cold chain risks such as transport delays and breakdowns, temperature abuse, and cross-contamination. Food losses are a burden to society, too, with one-third of all edible human food wasted annually.

It is well known that temperature variability is inevitable across the supply chain and that in the case of fresh products, degradation reactions are accelerated at temperatures above the recommended storage temperatures. Therefore, temperature variability must be accounted for when estimating the product's RSL at any point in the chain.

IoT makes it possible to measure environmental parameters such as temperature and humidity during transit and communicate these measurements in real-time to the cloud for quality monitoring and real-time fleet routing optimization. The product's RSL can be estimated in real time with a suitable computational method. A mathematical model that accepts the measured environmental parameters and outputs the

RSL for a given product in real time is thus pivotal to the success of such advanced decision systems. Predicting the shelf-life of sealed, dried, highly processed, and chemically-preserved foods is well understood and established with simple and reliable mathematical models. On the other hand, predicting the shelf-life of unsealed, unprocessed, and biologically active foods (such as fresh fruits and vegetables, milk, fish, and meat) shipped together with other types of fresh foods under chaotic logistic conditions remains a largely divisive and elusive problem. This is not due to the lack of complex mathematical models that cater to the main decay factors of such foods. Rather, the numerous limitations, high specificity to a food species, and the need for destructive tests and/or time-consuming pre-tests render current theoretical models unsustainable in practice.

In this paper, we present a new, real-time, non-destructive, highly generalizable computational method to predict the remaining shelf-life of FFVs under unplanned logistic conditions. The method uses real-time measurements of temperature and humidity and - most significantly - precludes the need for characterization pre-tests as it is initialized with only well-documented and easily accessible thermophysical

properties of the product. Moreover, the proposed model has been validated with three types of FFVs under various packaging conditions and frequencies of temperature disruptions.

II. LITERATURE SURVEY

During circulation, it cannot be guaranteed that the product's recommended storage temperature requirements will always be met. Consequently, the problem of shelf-life reduction due to deviation from the recommended storage conditions has long been of interest to the food science community. There are five types of decay in relation to FFVs: Preharvest biological decay, senescence, microbiological decay, chemical deterioration, and physical degradation. Despite the temperature and headspace gas composition being just two of many sources of degradation of fresh produce, they are still considered major predictors of post-harvest degradation and distribution losses. As a result, temperature and gas measurements can be used to provide an optimistic prediction of the RSL. All other sources of decay, such as vibration and moisture loss, can be ignored if they are not significantly different from the logistic conditions under which the initial shelf-life was estimated. Hence, for all future discussions, we shall use

the word 'decay' to refer to the amalgamation of all post-harvest, undesirable quality changes that are affected by temperature and/or atmospheric composition, namely: Senescence, chemical deterioration, and microbiological decay.

In their review, Hertog et al. categorized the modeling approaches for warehouse management into three groups, all of which use the measured environmental conditions along the cold chain. Namely, Statistical Process Control (SPC), specific quality attribute models, and generic shelf-life models. SPC models focus on monitoring and correcting environmental conditions, while specific attribute models monitor a single biochemical attribute over time. The most comprehensive approach is the generic shelf-life model, where the product's shelf-life is determined considering not only a limiting quality attribute of the product but also its respiration activity and moisture loss. We are interested in the second and third approaches, as they output an RSL estimate: An actionable piece of information based on which the storage, sales, and distribution of perishable products can be optimized, as opposed to a binary (Keep/Discard) recommendation from an SPC model.

Most well-known mathematical models of shelf-life traditionally fall into the second category (specific quality attribute models),

consisting of a primary kinetic model and one or more secondary models. The primary model consists of one or more differential equations relating the rate of change of the food quality index to time, while a secondary model relates the rate of change of the food quality index to degradation factors other than time, such as temperature, pH, water activity, etc. The evolution of the shelf-life (defined in terms of a food quality index) is found by integrating the differential equations in the primary and secondary models over time.

To develop a kinetic model of food degradation, a measurable physical, biological, chemical, or sensory property of the food that strongly correlates with loss of quality and customer acceptance, Q , is first defined, then experimentally measured for different storage temperatures and times. Afterward, the relationship between Q and storage time t at a constant temperature T is fitted with either a straight-line on a linear plot or a semi-log plot of the data points, whichever is the best fitting. The differential of this line is of either zero-order or first-order, based on which the overall kinetics of decay are classified as either zero-order or first-order kinetics, respectively, and the reaction rate constant k is found at that specific temperature T .

The reaction rate constant k is often a function of the storage temperature T , thus the need for a secondary model to account for the effect of temperature on the rate constant k .

The Arrhenius relationship has been widely used to model the reaction rate constant, k with respect to T . Eventually, a specific quality attribute model considers a product to have reached its shelf-life when the quality index Q reaches a predefined acceptability limit.

Despite the overwhelming popularity of using traditional zero-, first-, and second-order kinetics to model degradation in fresh and horticultural products, asserted that most food degradation processes do not follow any standard kinetic model and that shelf-life estimation need not assume a kinetic model order a priori. Another drawback of using fixed-order kinetics is that the change in the limiting quality attribute is often due to multiple biochemical reactions which are not necessarily of the same order.

PROPOSED SYSTEM

The overview of our proposed system is shown in the below figure.

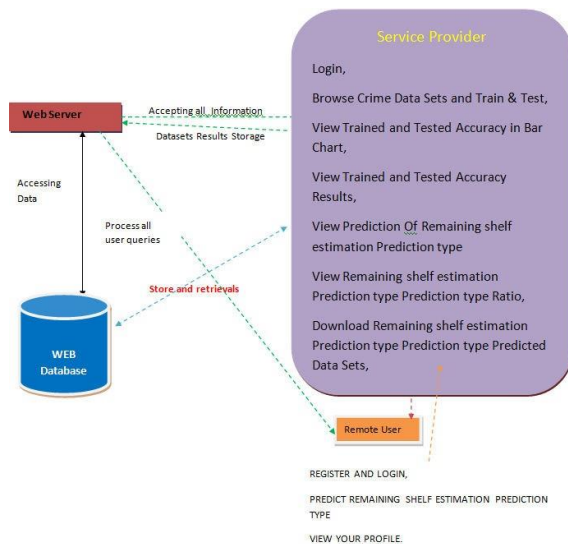


Fig. 1: System Overview

Implementation Modules

Service Provider

- In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as Browse the dataset, view trained and tested results, view predicted remaining shelf estimation prediction type, view predicted remaining shelf estimation prediction type ratio and view all remote users

Train and Test Model

- In this module, the service provider split the Used dataset into train and test data of ratio 70 % and 30 % respectively. The 70% of the data is consider as train data which is used to train the model and 30%

of the data is consider as test which is used to test the model

Remote User

- In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like predict remaining shelf estimation prediction type, View Your Profile.

Classification

- In this module, user enter data and classify them using tested machine learning models.

Implementation Algorithms

Support Vector Machine

- In machine learning, support-vector machines (SVMs, also support-vector networks) are supervised learning models with associated learning algorithms that analyze data for classification and regression analysis. An SVM training algorithm builds a model that assigns new examples to one category or the other, making it a non-probabilistic binary linear classifier.

Logistic Regression

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**

Naïve Bayes

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems.
- It is mainly used in text classification that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective

Classification algorithms which helps in building the fast machine learning models that can make quick predictions.

- It is a probabilistic classifier, which means it predicts on the basis of the probability of an object

IV. RESULTS



Fig. 2: Main Page

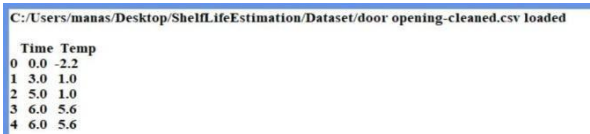


Fig. 3: Upload Dataset

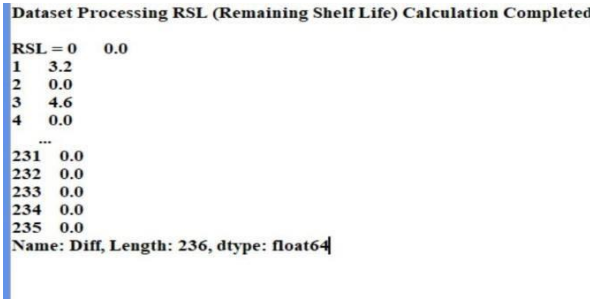


Fig. 4: Preprocess Data

Propose SLEM Model Error : 0.6408938280476446

Observed Life : 0 Days Predicted Life : 4 Days
 Observed Life : 9 Days Predicted Life : 1 Days
 Observed Life : 0 Days Predicted Life : 1 Days
 Observed Life : 13 Days Predicted Life : 10 Days
 Observed Life : 0 Days Predicted Life : 10 Days
 Observed Life : 7 Days Predicted Life : 15 Days
 Observed Life : 5 Days Predicted Life : 15 Days
 Observed Life : 6 Days Predicted Life : 14 Days
 Observed Life : 8 Days Predicted Life : 13 Days
 Observed Life : 15 Days Predicted Life : 12 Days

Fig. 5: Proposed Shelf Life Estimation+

ANOVA Graph

Propose SLEM Model Error : 0.6408938280476446

Observed Life : 0 Days Predicted Life : 4 Days
 Observed Life : 9 Days Predicted Life : 1 Days
 Observed Life : 0 Days Predicted Life : 1 Days
 Observed Life : 13 Days Predicted Life : 10 Days
 Observed Life : 0 Days Predicted Life : 10 Days
 Observed Life : 7 Days Predicted Life : 15 Days
 Observed Life : 5 Days Predicted Life : 15 Days
 Observed Life : 6 Days Predicted Life : 14 Days
 Observed Life : 8 Days Predicted Life : 13 Days
 Observed Life : 15 Days Predicted Life : 12 Days

Fig. 6: Proposed Shelf Life Estimation+

ANOVA Result

I. CONCLUSION

This project presented a white-box Shelf-Life Estimation Model (SLEM) implemented in Matlab, which uses any ambient temperature history to estimate the RSL of an FFV in real time. The proposed SLEM was validated experimentally for three fresh products in sealed and unsealed packaging under dynamic temperature profiles. The model performed well with unsealed strawberries and apricots, with errors ranging from 0.04 to 1.2 days in error. The model was exceptionally superior for strawberries,

an exemplary nonclimacteric fruit. A surrogate method (QnD) was proposed for computing the pulp temperature for fast, real-time applications, which also produced acceptable results for strawberries and apricots, but performed poorly for bunched and sealed spinach. Despite simplifying assumptions, the SLEM was found to generate statistically sound predictions nonetheless, which supports the view that the respiration rate and its interaction with time, temperature, and the local CO₂ concentration are the key predictors of decay, notwithstanding the effects of microbiological activity, relative humidity, and exogenous ethylene. Even though the original and surrogate model results are statistically sound, the surrogate model is not recommended for sealed fresh products or prolonged experiments if prediction errors exceeding 2 days are unacceptable by the end customer.

REFERENCE

- [1] S. K. Srivastava, A. Chaudhuri, and R. K. Srivastava, "Propagation of risks and their impact on performance in fresh food retail," *Int. J. Logistics Manag.*, vol. 26, no. 3, pp. 568–602, Nov. 2015.
- [2] J. Gustavsson, C. Cederberg, U. Sonesson, R. Van Otterdijk, and A.

- Meybeck, “Global food losses and food waste,” presented at the Save Food Congr., Düsseldorf, Germany, May 2011. Accessed: Jan. 6, 2022. [Online]. Available:
https://www.madr.ro/docs/ind-alimentara/risipa_alimentara/presentation_food_waste.pdf
- [3] Y. Bouzembrak, M. Kluche, A. Gavai, and H. J. P. Marvin, “Internet of Things in food safety: Literature review and a bibliometric analysis,” *Trends Food Sci. Technol.*, vol. 94, pp. 54–64, Dec. 2019. [Online]. Available:
<http://www.sciencedirect.com/science/article/pii/S0924224419303048>
- [4] M. Awad, M. Ndiaye, and A. Osman, “Vehicle routing in cold food supply chain logistics: A literature review,” *Int. J. Logistics Manag.*, vol. 32, no. 2, pp. 592–617, Apr. 2021, doi: 10.1108/IJLM-02-2020-0092.
- [5] G. Hough, *Sensory Shelf Life Estimation of Food Products*. Boca Raton, FL, USA: CRC Press, May 2010. [Online]. Available:
<https://www.taylorfrancis.com/books/9780429147180>
- [6] S. Jol, A. Kassianenko, K. Wszol, and J. Oggel, “Issues in time and temperature abuse of refrigerated foods,” *Food Saf.*, vol. 11, no. 6, pp. 30–35, 2006.
- [7] M. L. Hertog, I. Uysal, U. McCarthy, B. M. Verlinden, and B. M. Nicolai, “Shelflife modelling for first-expired-first-outwarehouse management,” *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 372, no. 2017, Jun. 2014, Art. no. 20130306. [Online]. Available:
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4006170/>
- [8] M. G. Corradini and M. Peleg, “Shelf-life estimation from accelerated storage data,” *Trends Food Sci. Technol.*, vol. 18, no. 1, pp. 37–47, Jan. 2007. [Online]. Available:
<https://www.sciencedirect.com/science/article/pii/S0924224406002421>
- [9] M. Peleg and M. G. Corradini, “Microbial growth curves: What the models tell us and what they cannot,” *Crit. Rev. Food Sci. Nutrition*, vol. 51, no. 10, pp. 917–945, Dec. 2011.
- [10] C. S. Barsa, M. D. Normand, and M. Peleg, “On models of the temperature

AUTHOR PROFILE

Ch. Rushyendra Mani has received MCA Degree from Audisankara College of Engineering and Technology (autonomous), Jawaharlal Nehru Technological University Anantapur in 2020 and 2 years of experience, currently working as assistant professor in MCA department.



M. Manasa is pursuing MCA from Audisankara College of Engineering and Technology (AUTONOMOUS) Gudur, Affiliated to JNTUA in 2024

CERTIFICATE OF PUBLICATION

This is to certify that the paper entitled

**"ESTIMATION OF SHELF LIFE OF FRESH FRUITS AND VEGETABLES USING
MACHINE LEARNING"**

Authored by

CH. RUSHYENDRA MANI, Asst. Professor

From

**Audisankara College of Engineering and Technology (AUTONOMOUS) Gudur,
Andhra Pradesh, India.**

Has been published in

JES JOURNAL, VOLUME 15, ISSUE 07, JULY - 2024



Dr. D. K. Umber
Editors-in-Chief
Dr. D. Karthikaran Umber
JES PUBLICATION



CERTIFICATE OF PUBLICATION

This is to certify that the paper entitled

**"ESTIMATION OF SHELF LIFE OF FRESH FRUITS AND VEGETABLES USING
MACHINE LEARNING"**

Authored by

M. MANASA

From

**Audisankara College of Engineering and Technology (AUTONOMOUS) Gudur,
Andhra Pradesh, India.**

Has been published in

JES JOURNAL, VOLUME 15, ISSUE 07, JULY - 2024



Dr. D. K. Chakrabarti
Editors-in-Chief
Dr. D. Karthikaran Umber
JES PUBLICATION





Elite

NPTEL Online Certification

(Funded by the MoE, Govt. of India)



This certificate is awarded to

MALLELA MANASA

for successfully completing the course

Introduction to Internet of Things

with a consolidated score of **70** %

Online Assignments	24.56/25	Proctored Exam	45/75
--------------------	----------	----------------	-------

Total number of candidates certified in this course: **32882**

Jan-Apr 2024

(12 week course)

Prof. Haimanti Banerji
Coordinator, NPTEL
IIT Kharagpur



Indian Institute of Technology Kharagpur



Roll No: NPTEL24CS35S663700885

To verify the certificate



No. of credits recommended: 3 or 4

THANK YOU